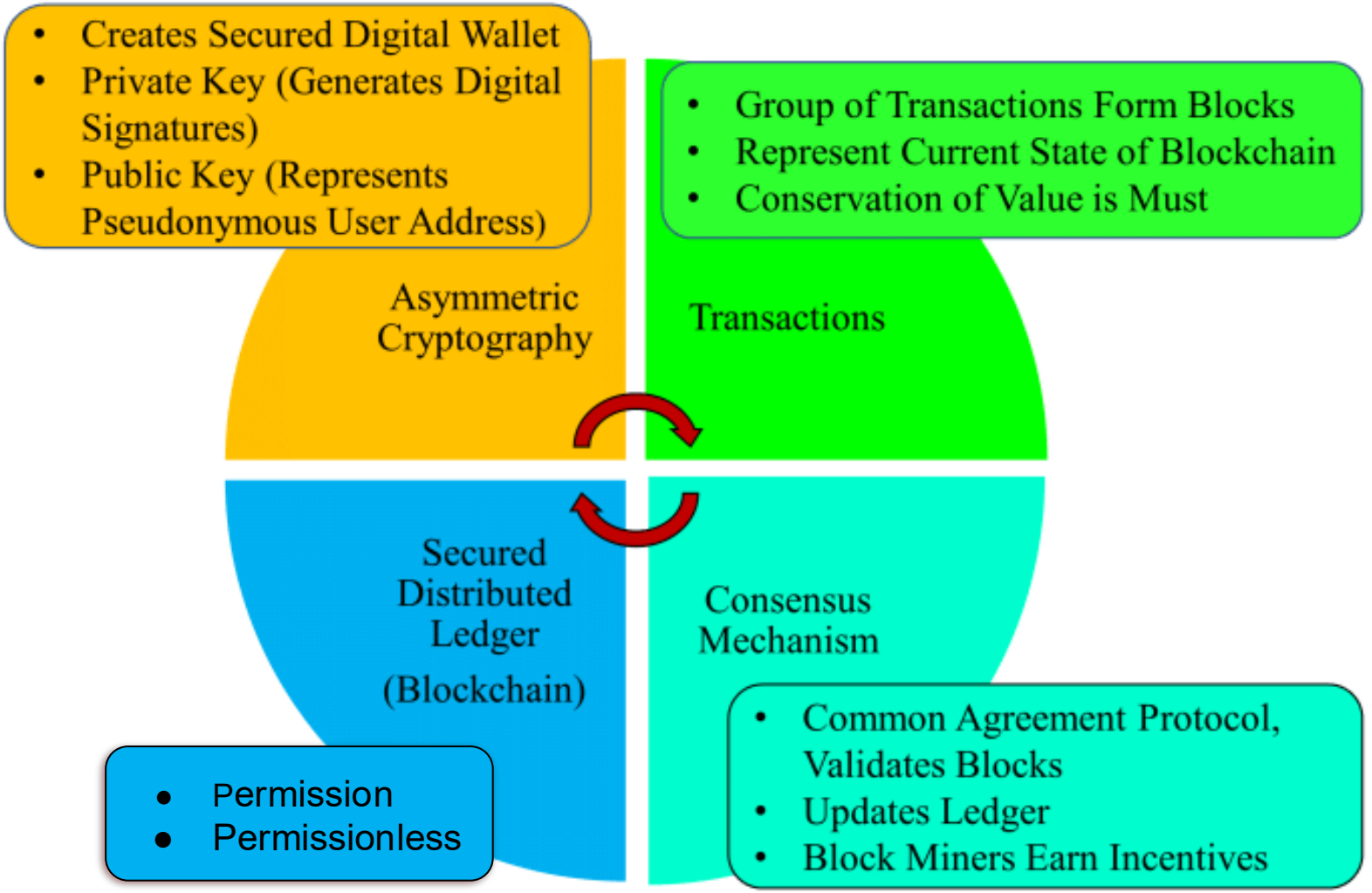
Blockchain Technology
BITS F452

BITS Pilani
K K Birla Goa Campus

Sanjay K. Sahay
Department of Computer Science and Information Systems

Blockchain Technology: Bitcoin

Core Components of Blockchain



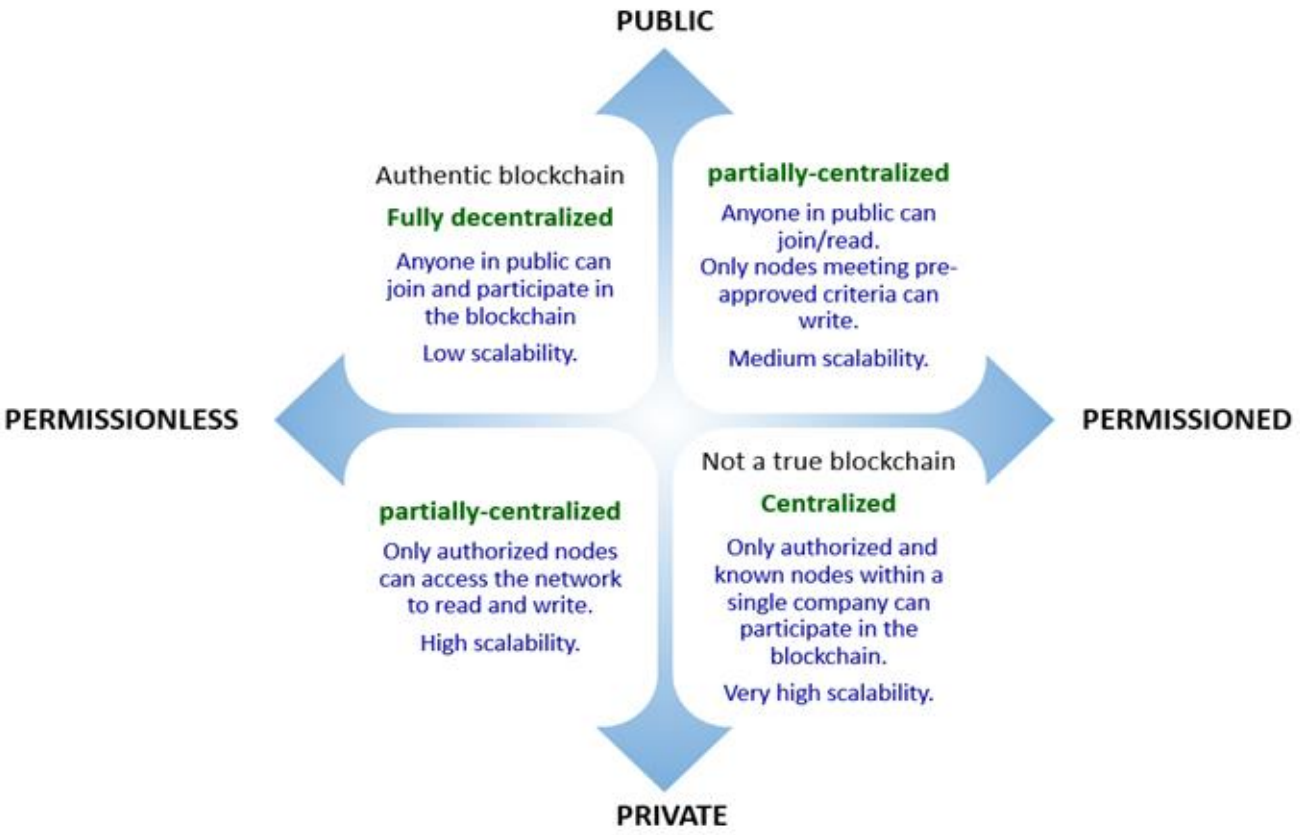
Blockchain: Introduction

- Block header: Time stamp, Version, Merkle root, Difficulty target, Nonce, Previous hash

Version
Previous Block Hash
Merkle Root
Timestamp
Difficulty Target
Nonce

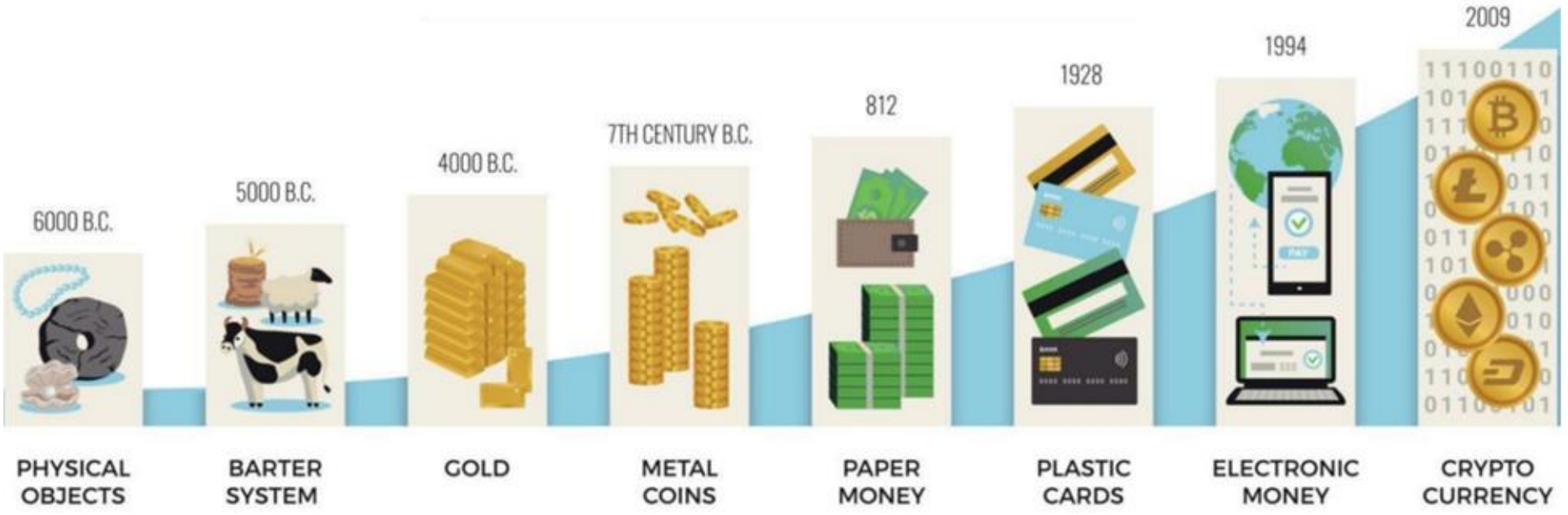
- Blockchain: Blocks, links, ledger, distribution, transactions, confirmations, PoW, results, Peer-to-peer network, cryptography, smart contract, consensus mechanisms.

Types of Blockchain



- TPS, Scalability, Trust, Security, Privacy, Participation, Central Authority, Open Source, Energy Consumption, Modification, Private Key, Risk of Corruption, Customization, Network operator , Identity and Access Management, Flexibility …

Evolution of Currency



- Cryptocurrency is a digital asset that is used as a medium of exchange on the blockchain.

Some Terms

- **Bitcoin:** A collection of concepts and technologies that form the basis of a digital money ecosystem. The name of the currency unit (the coin), the network and the software. Bitcoins are entirely virtual.
- **Node:** Computer (participant) within the system.
- **Block:** A grouping of transactions, marked with a timestamp, and a fingerprint of the previous block. The block header is hashed to find a proof-of-work, thereby validating the transactions. Valid blocks are added to the main blockchain by network consensus.
- **Consensus:** A mechanism through which networks reach the agreement.
- **Blockchain:** A list of validated blocks, each linking to its predecessor all the way to the genesis.
- **Difficulty:** A network-wide setting that controls how much computation is required to find a proof-of-work.

Some Terms

- **PoW:** A form of cryptographic proof in which one party (the prover) proves to others (the verifiers) that a certain amount of a specific computational effort has been done. Verifiers can subsequently confirm this expenditure with minimal effort on their part
- **Difficulty:** A network-wide setting that controls how much computation is required to find a proof-of-work.
- **Difficulty target:** A difficulty at which all the computation in the network will find blocks approximately every 10 minutes.
- **Difficulty retargeting:** A network-wide re-calculation of the difficulty which occurs once every 2106 blocks and considers the hashing power of the previous 2106 blocks.
- **BIP (Bitcoin Improvement Proposals)**: A set of proposals that members of the bitcoin community have submitted to improve bitcoin.

Some Terms

- **Genesis block:** The first block in the blockchain, used to initialize the crypto-currency.
- **Miner:** A network node that finds valid proof-of-work for new blocks, by repeated hashing.
- **Network:** A peer-to-peer network that propagates transactions and blocks to every bitcoin node on the network.
- **Reward:** An amount included in each new block as a reward by the network to the miner who found the proof-of-work solution. It is currently 3.125 BTC per block.

Some Terms

- **Transaction:** A transfer of bitcoins from one address to another. More precisely, a transaction is a signed data structure expressing a transfer of value. Transactions are transmitted over the bitcoin network, collected by miners and included into blocks, made permanent on the blockchain.
- **Wallet:** Software that holds all your bitcoin addresses and secret keys. Use it to send, receive and store your bitcoin.
- **Address or Public key:** A bitcoin address, just like you ask others to send an email to your email address, you would ask others to send you bitcoin to your bitcoin address.
- **Secret/Private key:** The secret number that unlocks bitcoins sent to the corresponding address.

Some Terms

- **Forking:** Process of duplication from an existing blockchain. It creates an alternative version of a blockchain, typically to upgrades or add/remove new governance rules to a network.
  - Hard fork: Not backward compatible, i.e., split in the blockchain Hence, all clients need to upgrade to the new version if they want to continue participating in the network. The old chain is eventually abandoned while, in other instances, both chains remain active, e.g., Bitcoin–Bitcoin Cash fork, Ethereum Classic–Ethereum fork.
  - Soft fork: Backward compatible protocol upgrade i.e., old software will recognize the blocks made by the new protocol as valid. Hence it does not require all the nodes in the network to upgrade to maintain consensus as the soft-forked chain follows both the old as well as the new set of protocol rules, e.g., Bitcoin protocol upgraded to SegWit. (Bitcoin network is the original BTC address while Bitcoin (SegWit) is the newer address format with lower fees)

Digital Currencies Before Bitcoin

- Late 1980s, researchers began trying to use cryptography to build digital currencies. These early digital currency projects issued digital money, usually backed by a national currency or precious metal such as gold.
- The emergence of viable digital money is closely linked to developments in cryptography. The fundamental questions for anyone accepting digital money are:
  - Can I trust the money is authentic and not counterfeit?
  - Can I be sure that no one else can claim that this money belongs to them and not me? (aka the “double-spend” problem)
- Issuers of paper money are constantly battling the counterfeiting problem by using increasingly sophisticated papers and printing technology. Physical money addresses the double-spend issue easily because the same paper note cannot be in two places at once.

Digital Currencies Before Bitcoin

- For digital money, which cannot take advantage of esoteric inks or holographic strips, cryptography provides the basis for trusting the legitimacy of a user's claim to value.
- Specifically, cryptographic digital signatures enable a user to sign a digital asset or transaction proving the ownership of that asset. With the appropriate architecture, digital signatures also can be used to address the double-spend issue.
- Early digital currencies used a central clearinghouse to settle all transactions at regular intervals, just like a traditional banking system.

Bitcoin Invention

- Satoshi Nakamoto combined several prior inventions such as b-money and HashCash to create a completely decentralized electronic cash system that does not rely on a central authority for currency issuance or settlement and validation of transactions.
- Invented in 2008 by Satoshi Nakamoto with the publication of a paper titled “Bitcoin: A Peer-to-Peer Electronic Cash System”. Bitcoin is the culmination of decades of research in cryptography and distributed systems and includes four key innovations brought together in a unique and powerful combination. Bitcoin consists of:
  - A decentralized peer-to-peer network (the bitcoin protocol)
  - A public transaction ledger (the blockchain)
  - A decentralized mathematical and deterministic currency issuance (distributed mining), and
  - A de-centralized transaction verification system

Bitcoin Invention

- Bitcoin is such a system, completely decentralized by design, and free of any central authority or point of control that can be attacked or corrupted.
- Bitcoin cannot be inflated beyond the expected issuance rate. Actually, it is deflationary (economic forces, policies, or assets that cause a sustained decrease in the general price level of goods and services, often increasing the purchasing power of money).
- Due to bitcoin's diminishing rate of issuance, over the long term, the bitcoin currency is deflationary.
- The protocol also halves the rate at which new bitcoins are created every 4 years, an limits the total number of bitcoins that will be created to a fixed total of 21 million coins.

Bitcoin Invention

- The key innovation was to use a distributed computation system (PoW) to conduct a global "election" every 10 minutes, allowing the decentralized network to arrive at consensus about the state of transactions.
- Solves the issue of double-spend where a single currency unit can be spent twice. Previously, the double-spend problem was a weakness of digital currency
- Invention is also a practical solution to a previously unsolved problem in distributed computing, known as the Byzantine Generals' Problem (problem consists of trying to agree on a course of action by exchanging information over an unreliable and potentially compromised network).

Bitcoin Invention

- Satoshi Nakamoto withdrew from the public in April of 2011, leaving the responsibility of developing the code and network to a thriving group of volunteers
- Neither Satoshi Nakamoto nor anyone else exerts control over the bitcoin system, which operates based on fully transparent mathematical principles.
- Users of bitcoin own keys which allow them to prove ownership of transactions in the bitcoin network, unlocking the value to spend it and transfer it to a new recipient. Those keys are often stored in a digital wallet on each user's computer. Possession of the key that unlocks a transaction is the only prerequisite to spending bitcoins, putting the control entirely in the hands of each user
- To join the bitcoin network and start using the currency, all a user has to do is download an application or use a web application.

Bitcoin Client

- A Bitcoin client is a software that interacts directly with the Bitcoin network, enabling users to perform various functions like creating wallets, managing transactions, and viewing the blockchain.
- **Full Client:** A full client/ node is a client that stores the entire history of bitcoin transactions, manages the user's wallets and can initiate transactions directly on the bitcoin network i.e., it can handles all aspects of the protocol without relying on any other servers or third party services.
- **Light Client:** A lightweight client stores the user's wallet but relies on third-party owned servers for access to the bitcoin transactions and network. It does not store a full copy of all transactions and therefore must trust the third party servers for transaction validation.
    - **SPV** (Simplified Payment Verification) clients, only download essential parts of the blockchain, making them less resource-intensive and suitable for mobile and low-power devices.

Bitcoin Client

- **Web Client:** Web-clients are accessed through a web browser and store the user's wallet on a server owned by a third party.
- **Mobile Bitcoin:** Mobile clients for smartphones are based on the Android system, can either operate as full clients, light clients or web clients. Some mobile clients are synchronized with a web or desktop client, providing a multi-platform wallet across multiple devices but with a common source of funds.
- Popular Bitcoin clients include Bitcoin Core (the original software implementation), Electrum, and Wasabi Wallet. These programs are crucial tools for those who wish to use Bitcoin, allowing them to manage their cryptocurrency safely and efficiently.

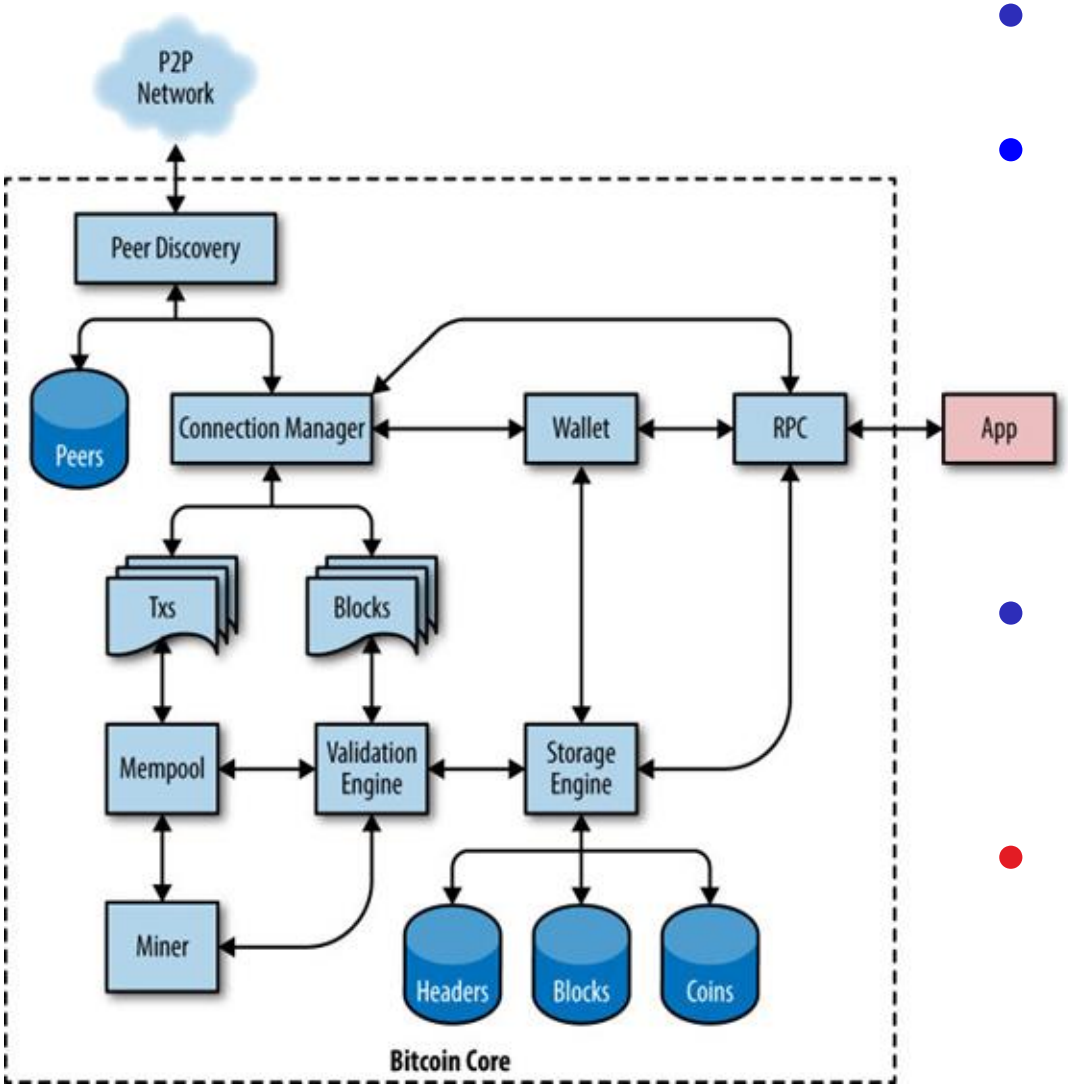
.

Coins and Tokens

- Coins include Bitcoin and altcoins (all cryptocurrencies other than Bitcoin).
- An Altcoin is an alternative digital currency to Bitcoin which refers to a group of cryptocurrencies.
- Tokens are digital units of value built on top of an existing blockchain programmable assets that live within the blockchain of a given platform. It provides access to a dApp or facilitating transactions on that blockchain.

COINS	TOKENS
Operates on its own blockchain.	Does not have its own blockchain.
Digital currency for payments similar to USD.	Created on another blockchain through a contract.
Can create its own protocols	Anyone can create a token.

Bitcoin Core Architecture

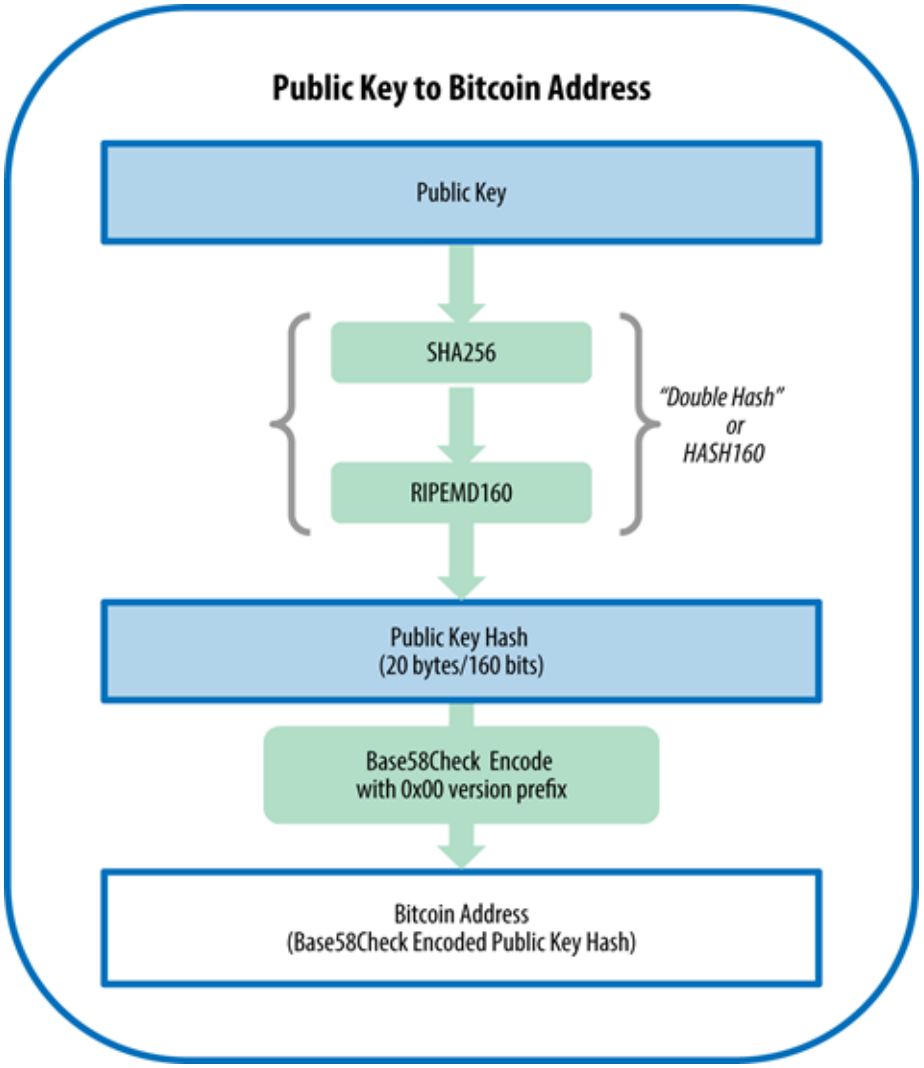


- Wallets allows users to receive and send cryptocurrency.
- The term "wallet" is a misnomer, as real money is not stored i.e. **A** sends cryptocurrency coins to **B's** wallet by signing off ownership of the coins, and **B** can spend the coins only if the private key is stored in **B's** wallet matches the public address of the assigned cryptocurrency.
- No real coins, the transaction is a new record added on the blockchain along with a change in the balance in the cryptocurrency wallet.
- The wallet provides all the features that are needed for a safe, easy and secure transfer of funds between two parties, namely, privacy, secure transactions, ease of usage, currency conversion.

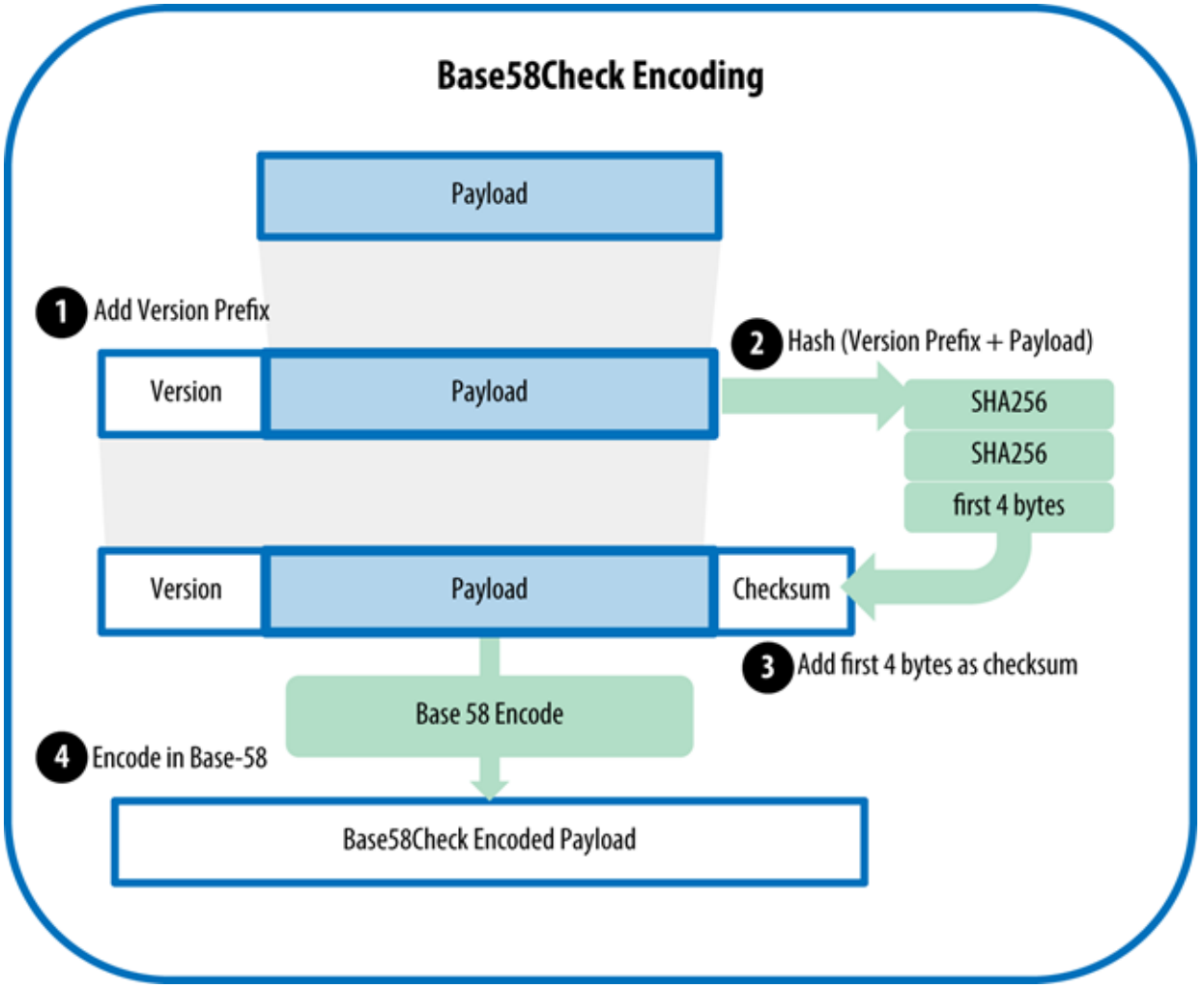
Transaction: Address

- Ownership of BTC is established through digital keys, BTC addresses and digital signatures
- Digital keys are not stored in the network, instead created and stored by end-users in a file, or simple database, called a wallet.
- Transaction requires a valid signature to be included in the blockchain, which can only be generated with valid digital keys.
- In a BTC transaction/payment, the recipient's public key is represented by its digital fingerprint, called a bitcoin address (Cheque).
- Addresses produced from public keys consist of a string of numbers and letters, beginning with the digit "1".

Transaction: Address



Encoding BTC Data



1J7mdg5rbQyUHENYdx39WVWK7fsLpEoXZy

Base58Check Version Prefix

- Base58Check is a Base-58 encoding format, which has a built-in error-checking code to add extra security against typos or transcription errors, The checksum is an additional four bytes added to the end of the data that is being encoded

- BTC address prefix is zero (0x00 in hex), whereas the prefix used when encoding a private key is 128 (0x80 in hex).

Type	Version prefix (hex)	Base-58 result prefix
Bitcoin Address	0x00	1
Pay-to-Script-Hash Address	0x05	3
Bitcoin Testnet Address	0x6F	m or n
Private Key WIF	0x80	5, K or L
BIP38 Encrypted Private Key	0x0142	6P
BIP32 Extended Public Key	0x0488B21E	xpub

Private Key Formats

Type	Prefix	Description
Hex	None	64 hexadecimal digits
WIF	5	Base58Check encoding: Base-58 with version prefix of 128 and 32-bit checksum
WIF-compressed	K or L	As above, with added suffix 0x01 before encoding

Format	Private Key
Hex	1E99423A4ED27608A15A2616A2B0E9E52CED330AC530EDCC32C8FFC6A526AEDD
WIF	5J3mBbAH58CpQ3Y5RNJpUKPE62SQ5tfcvU2JpbnkeyhfsYB1Jcn
Hex-compressed	1E99423A4ED27608A15A2616A2B0E9E52CED330AC530EDCC32C8FFC6A526AEDD_01_
WIF-compressed	KxFC1jmwwCoACiCAWZ3eXa96mBM6tb3TYzGmf6YwgdGWZgawvrtJ

Compressed and Encrypted Private Key

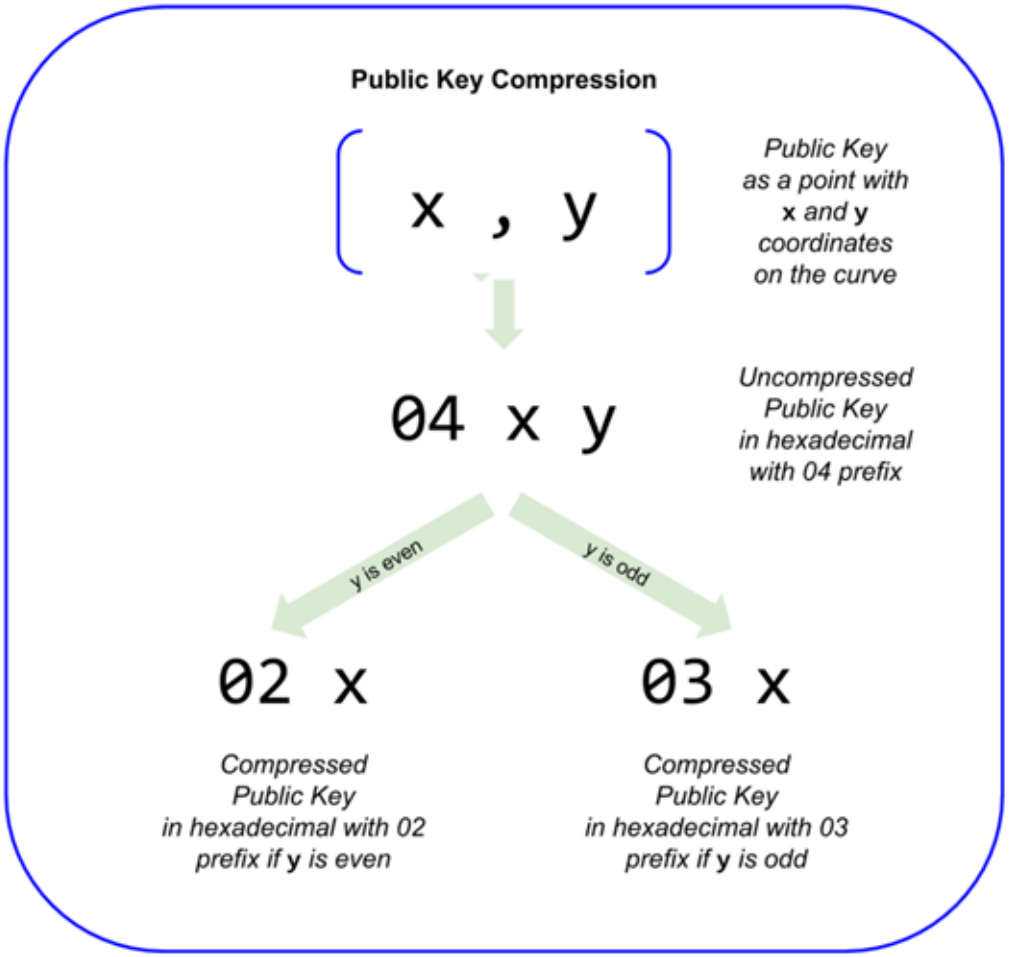
- Private keys are not compressed and cannot be compressed. Compressed private key" is misleading, because when a private key is exported as WIF-compressed it is actually one byte longer than an "uncompressed" private key. That is because it has the added 01 suffix which signifies it comes from a newer wallet and should only be used to produce compressed public keys.

- The term "compressed private key" really means "private key from which compressed public keys should be derived", whereas "uncompressed private key" really means "private key from which uncompressed public keys should be derived".

- Private keys are encrypted with a passphrase and encoded with Base58Check so that it can be stored securely on backup media, transported securely between wallets or otherwise (BIP0038 encryption scheme is a Base58Check encoded encrypted private key that begins with the prefix 6P)

Public Key Format

- Either compressed or uncompressed public keys.
- Public Key K defined as a point K = (x,y) = (F028892BAD...DC341A, 07CF33DA18...505BDB)
- Uncompressed Public Key K shown in hex (130 hex digits) as 04xy. K = 04F028892BAD...505BDB
- Compressed public keys start with either a 02 or a 03 prefix.



P2SH

- Bitcoin addresses that begin with the number “3” are pay-to-script-hash (P2SH) addresses, sometimes erroneously called multi-signature or multi-sig addresses, unlike transactions that “send” funds to traditional “1” bitcoin addresses, also known as pay-to-public-key-hash (P2PKH). It require more than the presentation of one public key hash and one private key signature as proof of ownership.

- It designate the beneficiary of a bitcoin transaction as the hash of a script, instead of the owner of a public key.

- The most common implementation of the P2SH function is the multi-signature address script. As the name implies, the underlying script requires more than one signature to prove ownership and therefore spend funds. The bitcoin multi-signature feature is designed to require M signatures (also known as the “threshold”) from a total of N keys, known as an M-of-N multi-sig, where M is equal to or less than N.

Vanity Addresses

- Vanity addresses are valid bitcoin addresses that contain human-readable messages, for example 1LoveBPzzD72PUXLzCkYAtGFYmK5vYNR33 is a valid address that contains the letters forming the word “Love” as the first four Base-58 letters

- Require generating and testing billions of candidate private keys, until one derives a bitcoin address with the desired pattern.

- Vanity addresses are no less or more secure than any other address, depend on the same ECC and SHA as any other address.

- Require to test billions of candidate private keys, until one derives a bitcoin address with the desired pattern.

Vanity Addresses

- The frequency of a vanity pattern (1KidsCharity)

Length	Pattern	Frequency	Average Search Time
1	1K	1 in 58 keys	< 1 milliseconds
2	1Ki	1 in 3,364	50 milliseconds
3	1Kid	1 in 195,000	< 2 seconds
4	1Kids	1 in 11 million	1 minute
5	1KidsC	1 in 656 million	1 hour
6	1KidsCh	1 in 38 billion	2 days
7	1KidsCha	1 in 2.2 trillion	3-4 months
8	1KidsChar	1 in 128 trillion	13-18 years
9	1KidsChari	1 in 7 quadrillion	800 years
10	1KidsCharit	1 in 400 quadrillion	46,000 years
11	1KidsCharity	1 in 23 quintillion	2.5 million years

Vanity Addresses Security

- Vanity addresses can be used to enhance and to defeat security measures, they are truly a double-edged sword. Used to improve security, a distinctive address makes it harder for adversaries to substitute their own address and fool your customers into paying them instead of you. Unfortunately, vanity addresses also make it possible for anyone to create an address that resembles any random address, or even another vanity address, thereby fooling your customers.

- Generating vanity addresses to match a random address:

- Original Random Address: 1J7mdg5rbQyUHENYdx39WVWK7fsLpEoXZy

- Vanity (4 character match) 1J7md1QqU4LpctBetHS2ZoyLV5d6dShhEy
- Vanity (5 character match) 1J7mdgYqyNd4ya3UEcq31Q7sqRMXw2XZ6n
- Vanity (6 character match) 1J7mdg5WxGENmwyJP9xuGhG5KRzu99BBCX

Wallets

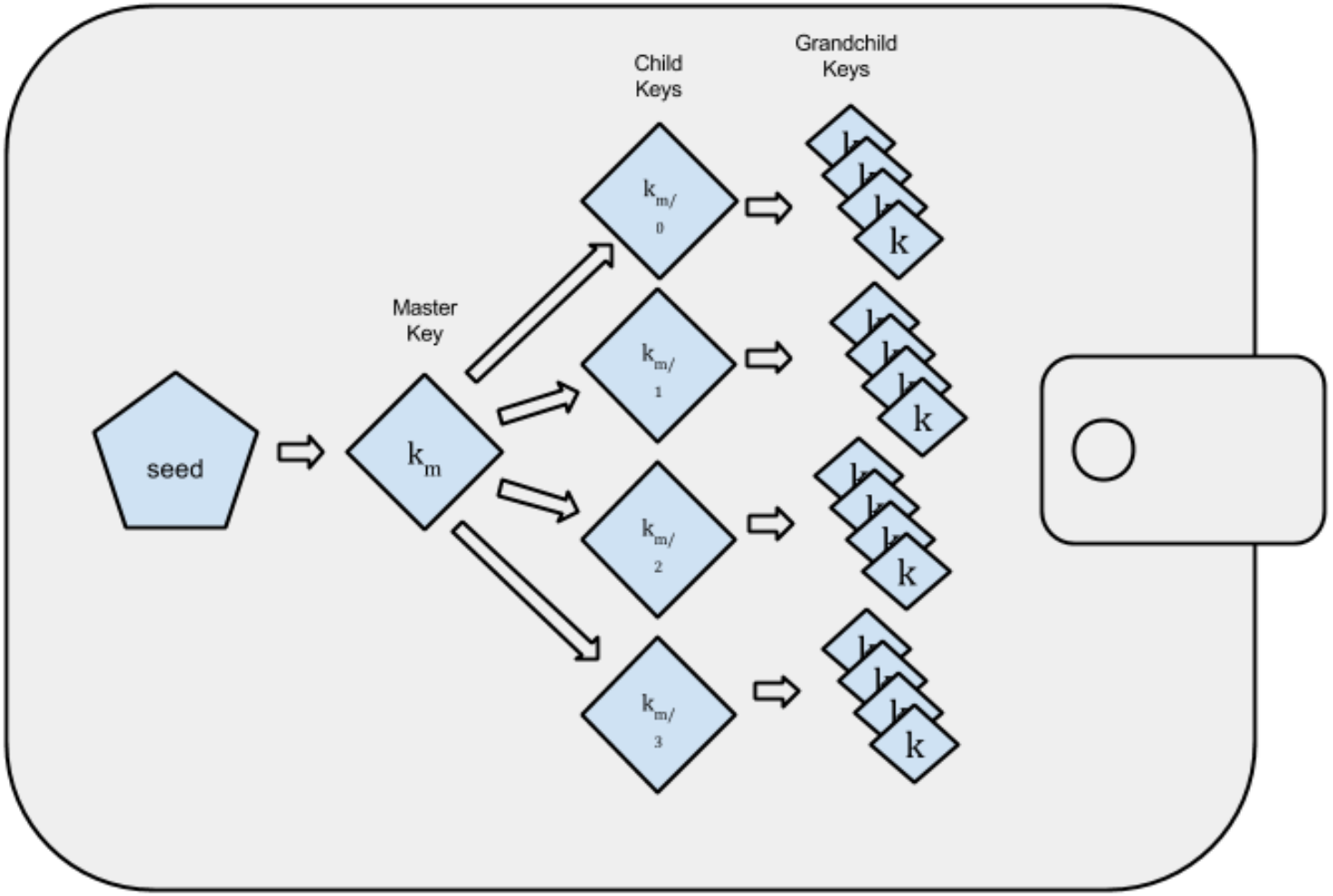
- Create public keys to receive BTC/satoshis and use the corresponding private keys to spend BTC.
- Store private keys and (optionally) other information related to transactions for the wallet program.
- Permitting receiving and spending of satoshis is the only essential feature of wallet software.
- Wallet program doesn't need to do both things i.e., two wallet programs can work together, one program. distributing public keys in order to receive satoshis and another program signing transactions spending those satoshis.

Wallets

- Non-deterministic (Random) - “Just a Bunch of Keys” (JBOK)
- Deterministic Wallets (Seeded)
- Mnemonic Code Words (Words to seed then key)
  - Create a random sequence of 128 to 256 bits.
  - Create a checksum of the random sequence by taking the first few bits of its SHA256 hash and then add the checksum to the end of the random sequence.
  - Divide the sequence into sections of 11 bits, using those to index a dictionary of 2048 pre-defined words.
  - Produce 12-24 words representing the mnemonic code.

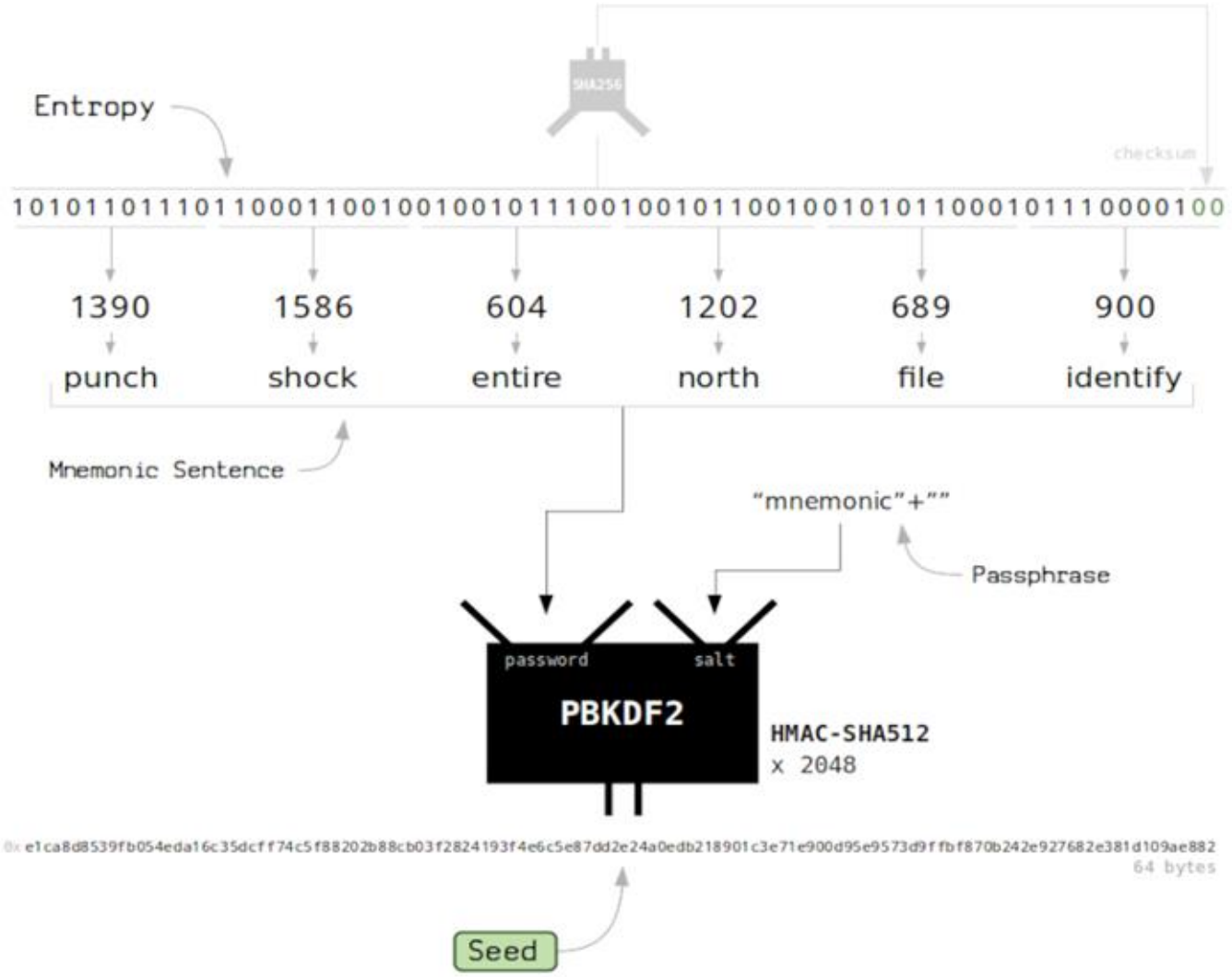
Entropy (bits)	Checksum (bits)	Entropy+Checksum	Word Length
128	4	132	12
160	5	165	15
192	6	198	18
224	7	231	21
256	8	264	24

Hierarchical Deterministic Wallets

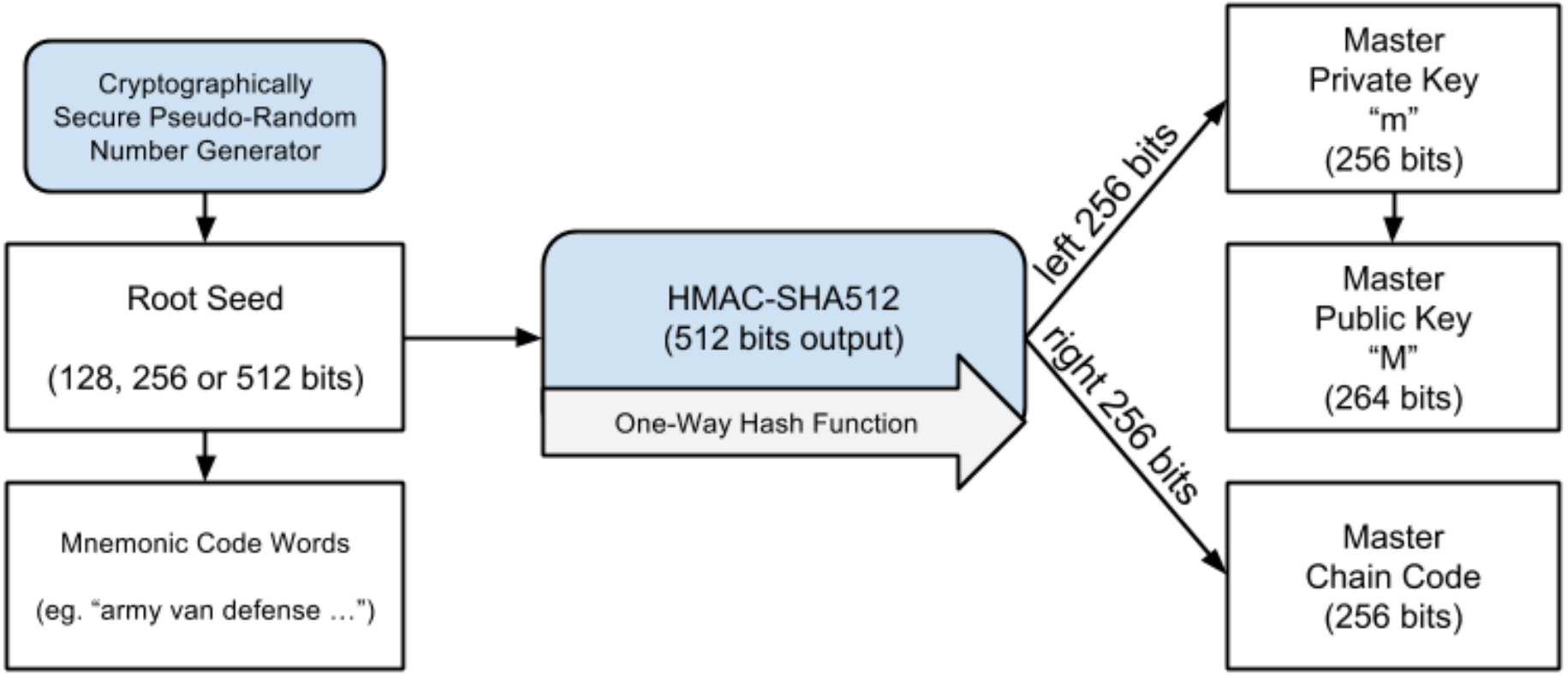


Type-2 Hierarchical Deterministic Wallet: A Tree of Keys Generated from a Seed

Seed Generation



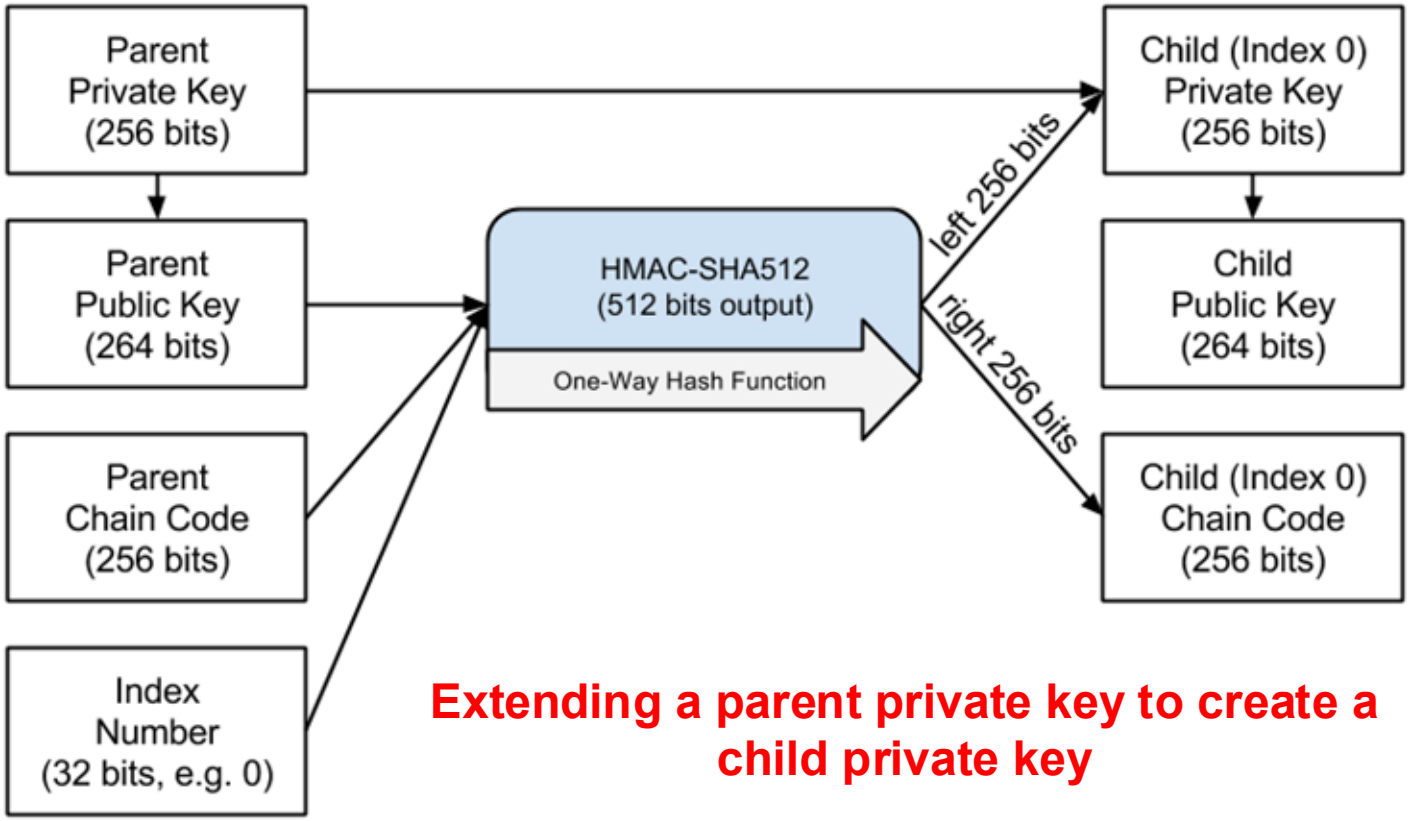
HD Wallet Creation From a Seed



Creating master keys and chain code from a root seed

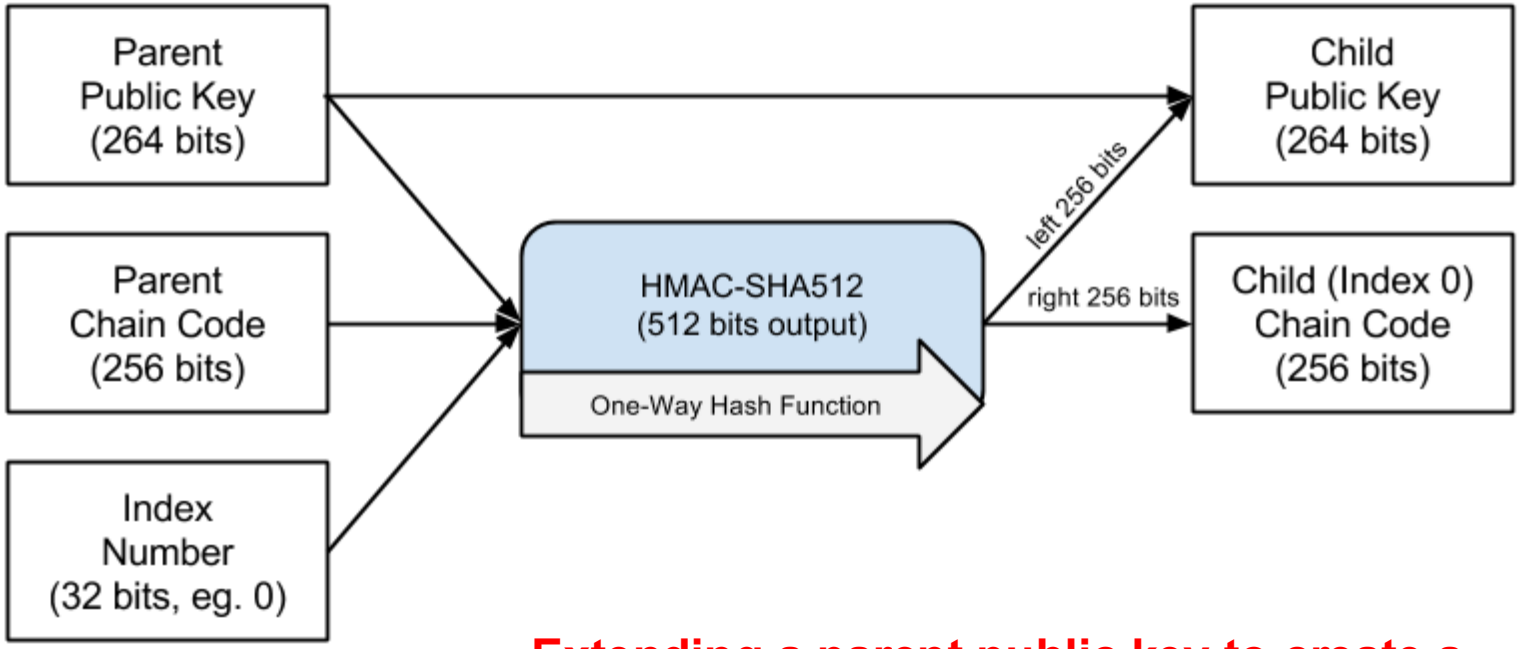
Private Child Key Derivation

- HD wallets use child key derivation (CKD) function to derive children keys from parent keys using hash functions that combines a parent private or public key, a seed called a chain code and an index number (32 bits)



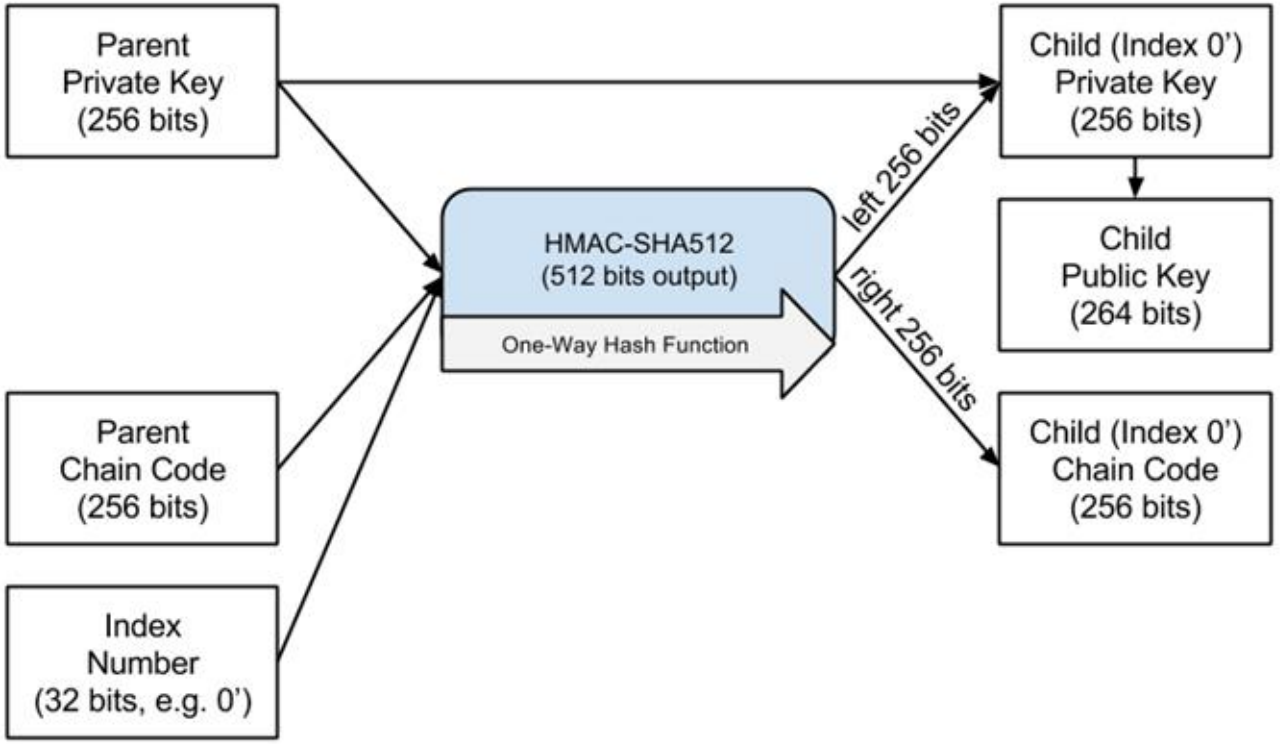
Extending a parent private key to create a child private key

Public child key derivation



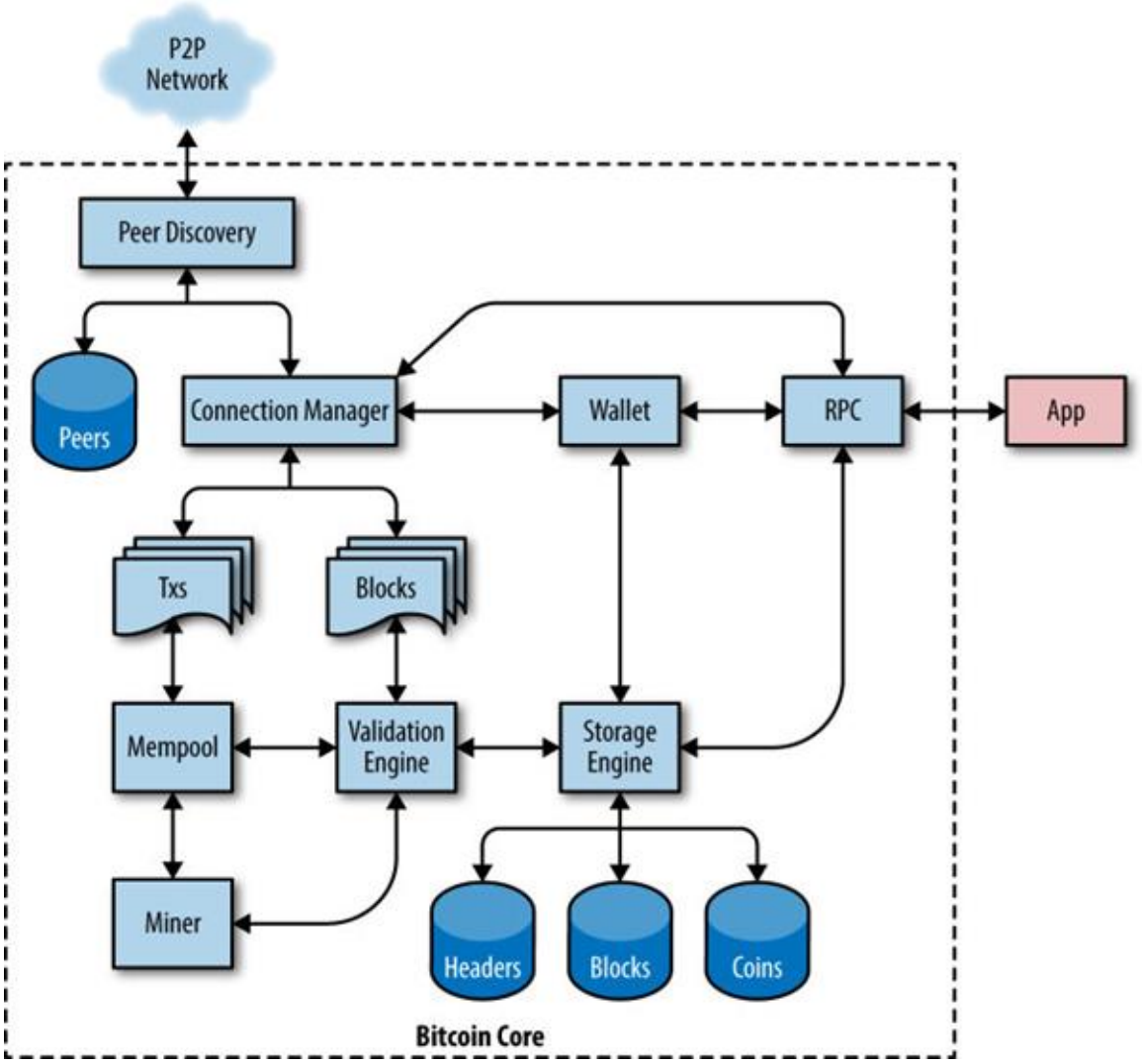
Extending a parent public key to create a child public key

Hardened child key derivation

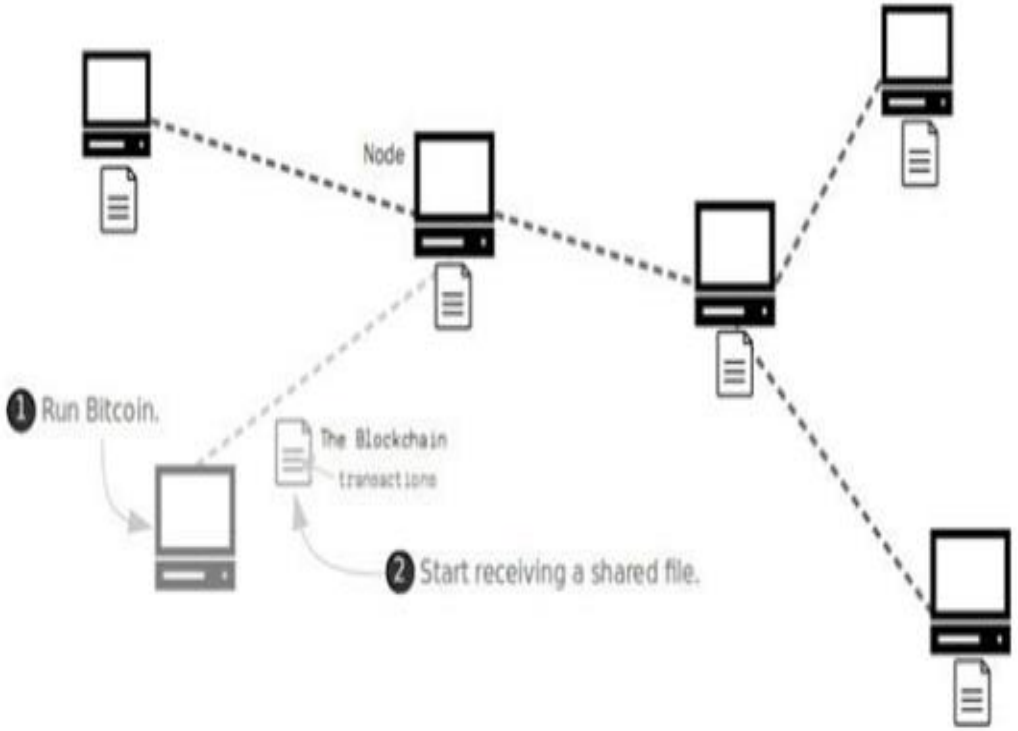


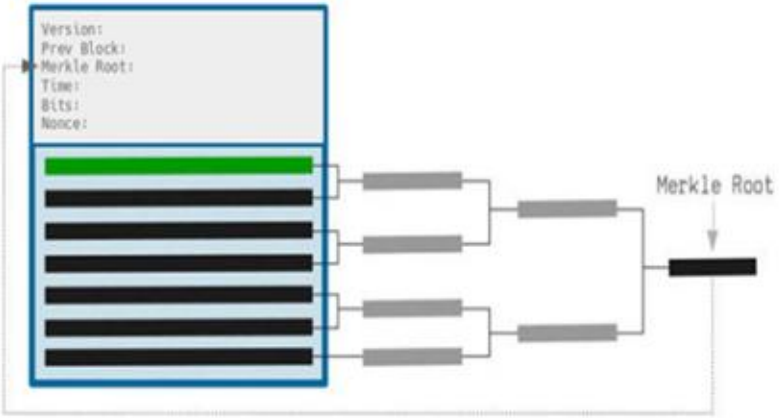
- Hardened derivation of a child key, omits the parent public key
- 0 to 2^{31} - 1 used only for normal derivation 2^{31} to 2^{32} – 1 for hardened derivation

Bitcoin: Transaction



Transactions Initiated by a Node





- Transactions are the most essential part of BTC system
- Transactions are data structures that encode the transfer of value between participants in the BTC system
- Each transactions is a public entry in BTC blockchain, the global double-entry bookkeeping ledger

Structure of a Block

- The blockchain data structure is an ordered back-linked list of blocks of transactions, and can be stored as a flat file, or in a simple database (BTC core client stores the blockchain metadata using Google's LevelDB database)

- A block has just one parent, however it can temporarily have multiple children. Each of the children refers to the same block as its parent and contains the same (parent) hash in the "previous block hash" field.

- Sequence of hashes linking each block to its parent, creates a chain going back all the way to the first block ever created, known as the genesis block.

- The block is made of a header, containing metadata, followed by a long list of transactions that make up the bulk of its size. The block header is 80 bytes, whereas the average transaction is at least 250 bytes and the average block contains more than 500 transactions.

Structure of a Block

Size	Field	Description
4 bytes	Block Size	The size of the block, in bytes, following this field
80 bytes	Block Header	Several fields form the block header (see below)
1-9 bytes (VarInt)	Transaction Counter	How many transactions follow
Variable	Transactions	The transactions recorded in this block

Size	Field	Description
4 bytes	Version	A version number to track software/protocol upgrades
32 bytes	Previous Block Hash	A reference to the hash of the previous (parent) block in the chain
32 bytes	Merkle Root	A hash of the root of the Merkle-Tree of this block's transactions
4 bytes	Timestamp	The approximate creation time of this block (seconds from Unix Epoch)
4 bytes	Difficulty Target	The proof-of-work algorithm difficulty target for this block
4 bytes	Nonce	A counter used for the proof-of-work algorithm

Block Identifiers: Hash and Height

- The block hash of the first bitcoin block ever created is 000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f

  https://blockchain.info/block/000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f

- Block hash identifies a block uniquely and unambiguously and can be independently derived by any node by simply hashing the block header.

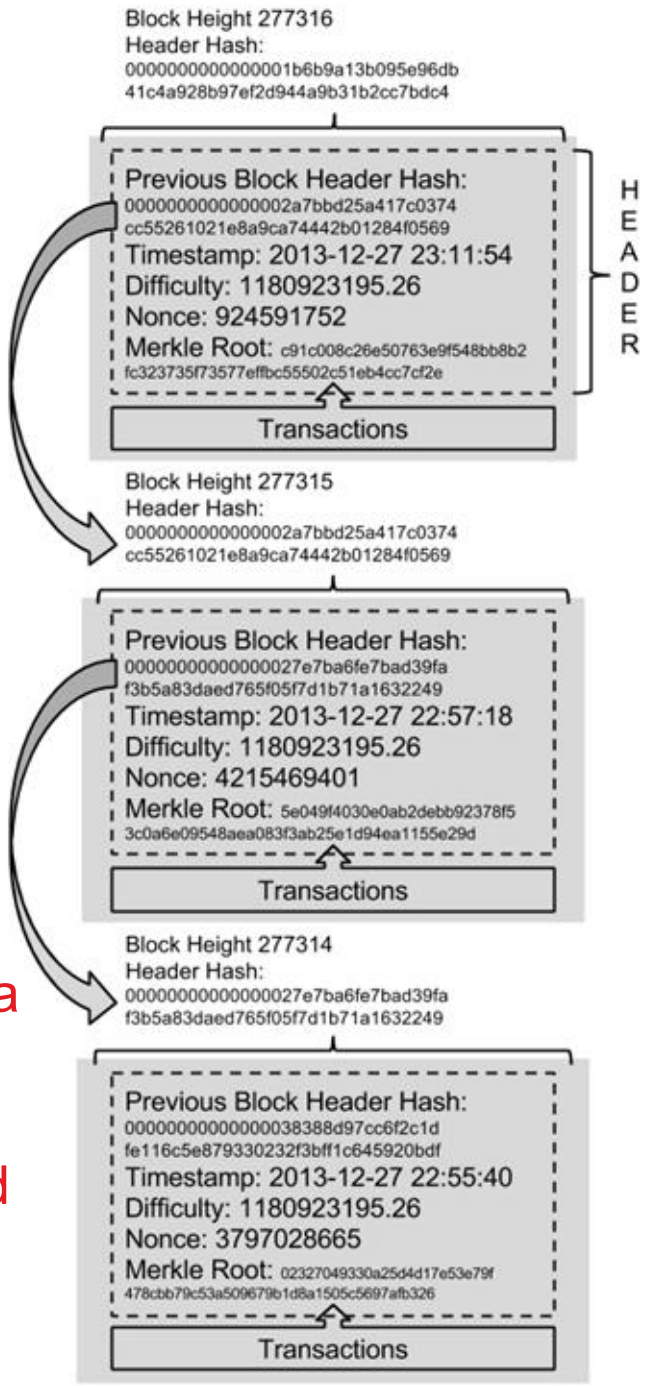
- A second way to identify a block is by its position in the blockchain, called the block height. The first block ever created is at block height 0 (zero) and is the same block that was referenced by the block hash.

- It is not always the case that a specific block height can identify a single block i.e. two or more blocks may compete for a single position in the blockchain.

Genesis Block

- The genesis block contains a hidden message within it. The coinbase transaction input contains the text “The Times 03/Jan/2009 Chancellor on brink of second bailout for banks” (embedded by Satoshi Nakamoto. This message provides proof of the earliest date this block was created, by referencing the headline of the british newspaper The Times.

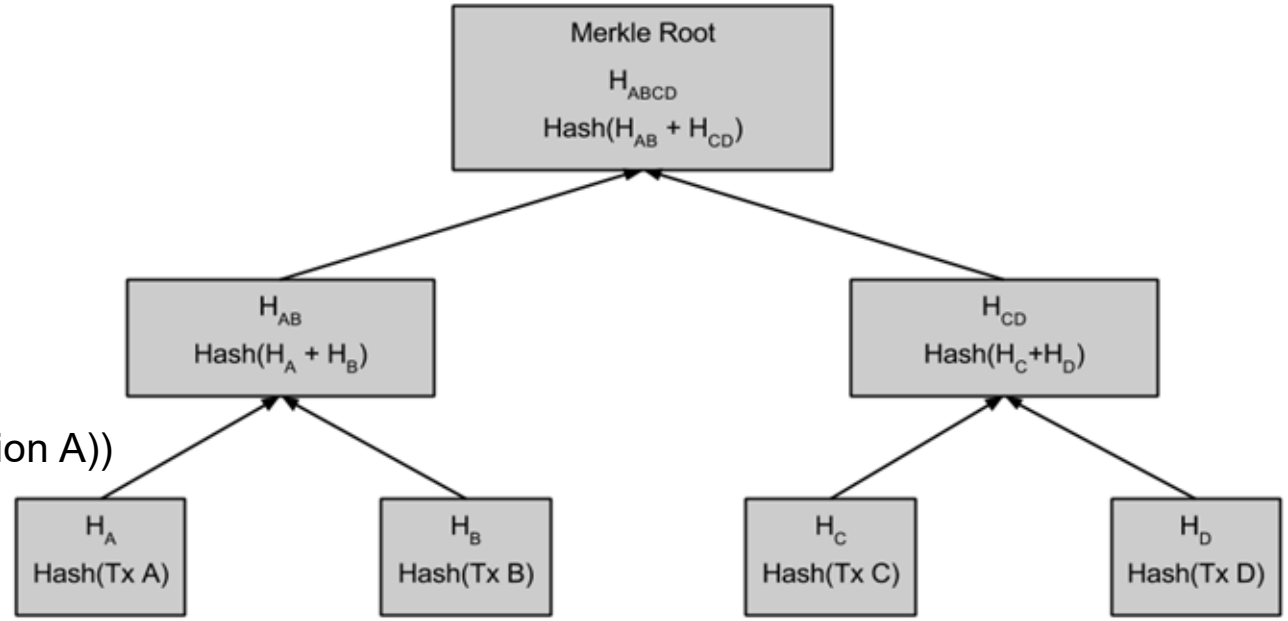
- A coinbase transaction, also known as a generation transaction, is the first transaction in a newly mined block viz. BTC, created by miners and serves to generate new cryptocurrency as a reward for their work in securing the network and adding a new block to the blockchain.



Merkle Tree

- A Merkle Tree, also known as a Binary Hash Tree is a data structure used for efficiently summarizing and verifying the integrity of large sets of data.

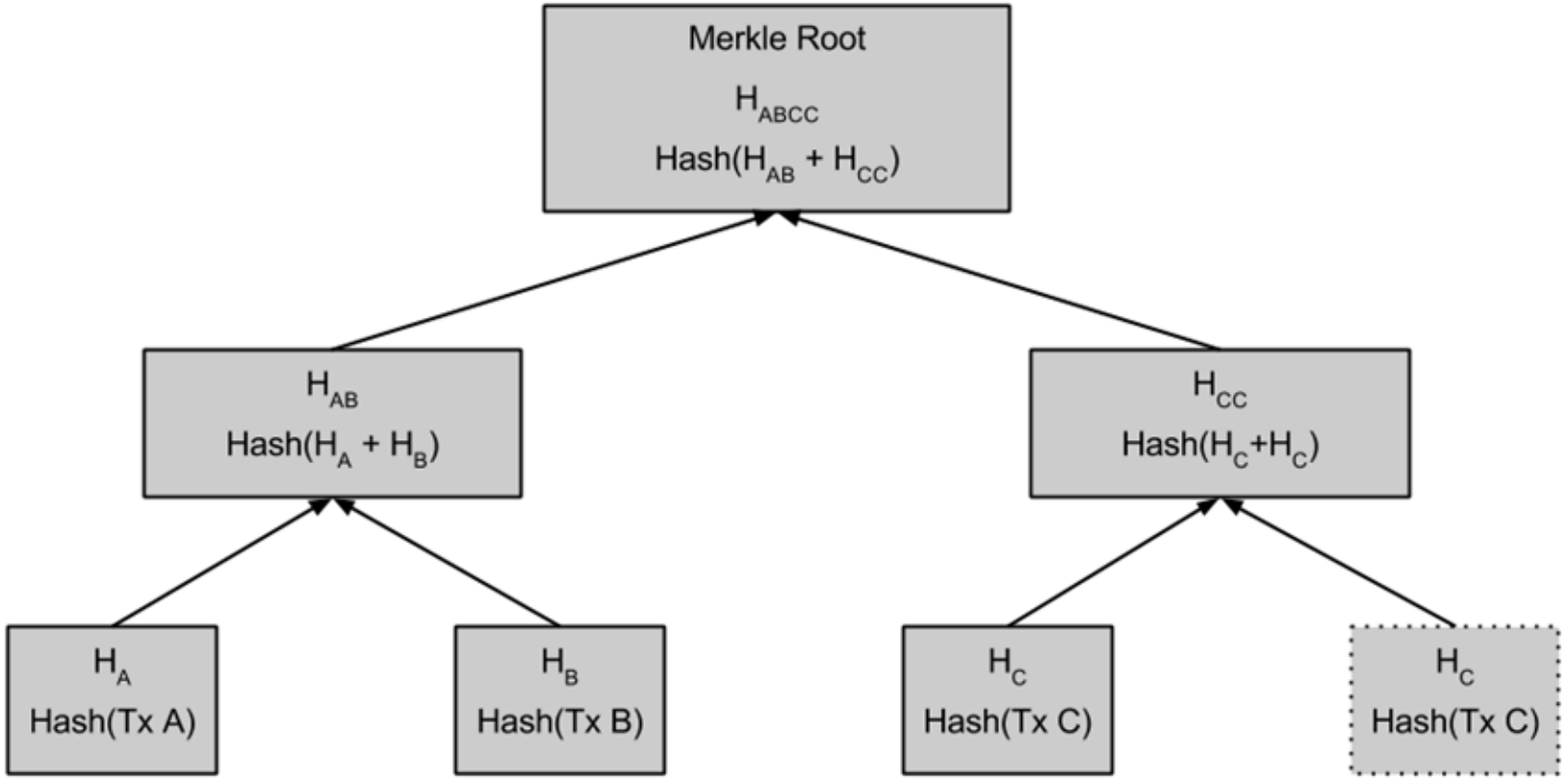
- Tree is constructed by recursively hashing pairs of nodes until there is only one hash, called the root, or merkle root. The cryptographic hash algorithm used in BTC is SHA256 applied twice, aka double-SHA256



H_A = SHA256(SHA256(Transaction A))

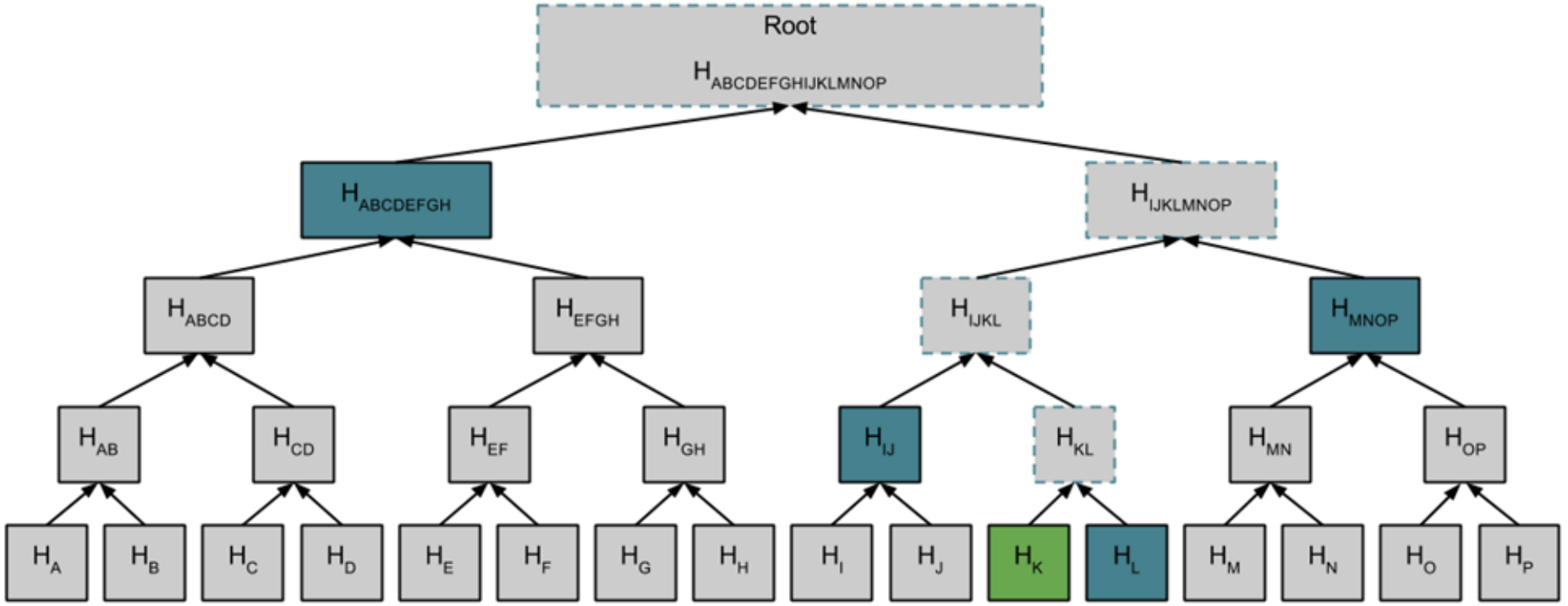
Merkle Tree

- Merkle tree is a binary tree, it needs an even number of leaf nodes. If there is an odd number of transactions to summarize, the last transaction hash will be duplicated to create an even number of leaf nodes, also known as a balanced tree



Merkle Tree Efficiency

- Merkle Path used to prove inclusion of a data element



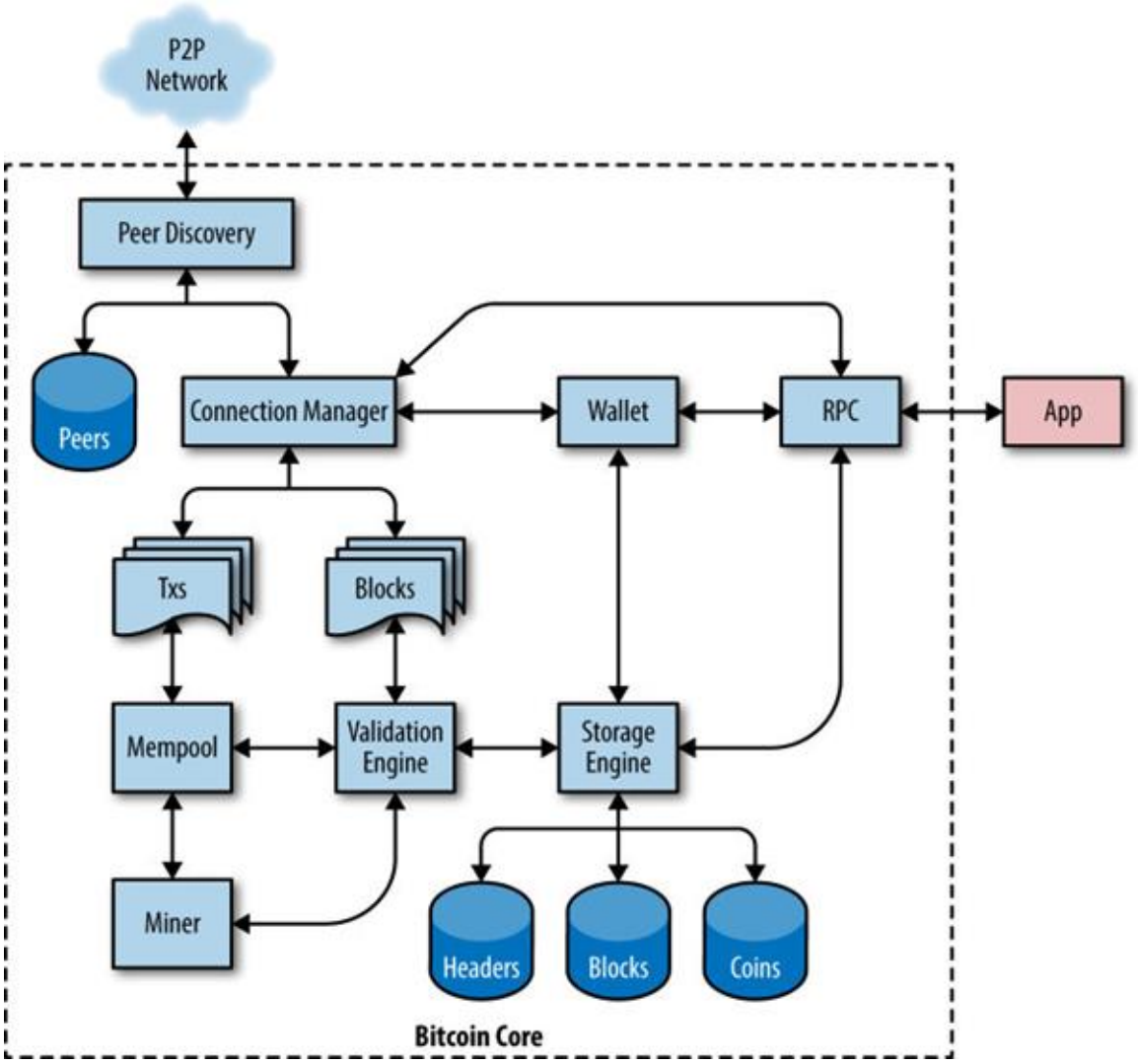
Merkle Tree Efficiency

Number of Transactions	Approx. Size of Block	Path Size (Hashes)	Path Size (Bytes)
16 transactions	4 kilobytes	4 hashes	128 bytes
512 transactions	128 kilobytes	9 hashes	288 bytes
2048 transactions	512 kilobytes	11 hashes	352 bytes
65,535 transactions	16 megabytes	16 hashes	512 bytes

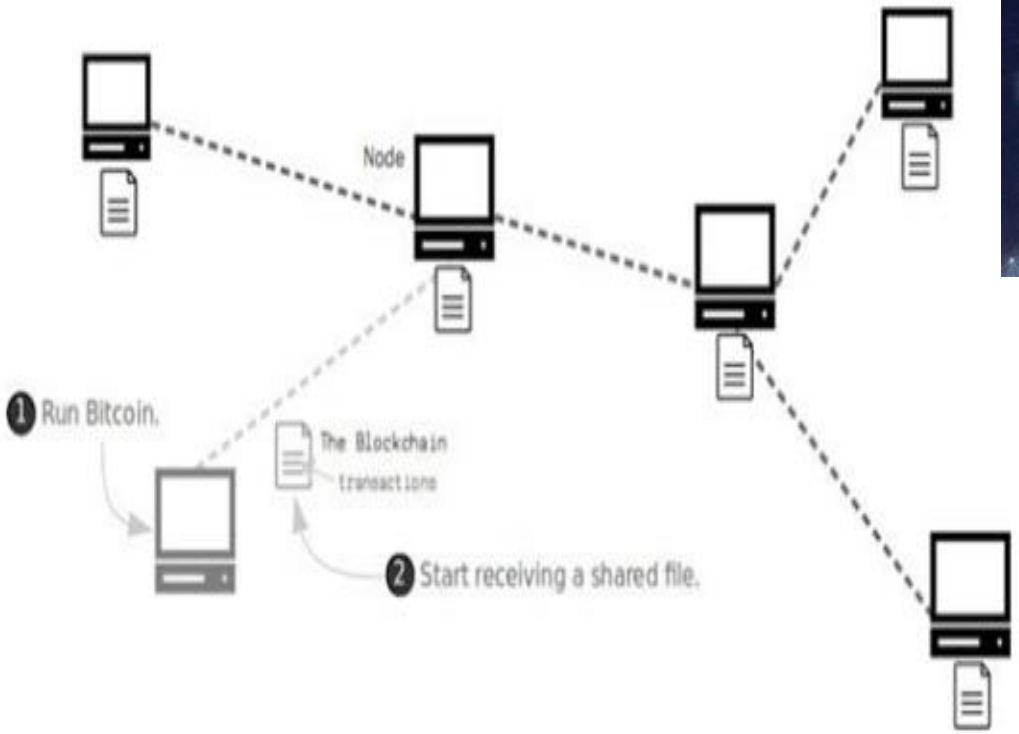
- Simplified Payment Verification: To verify that a transaction is included in a block, without having to download all the transactions in the block, they use an authentication path, or merkle path.

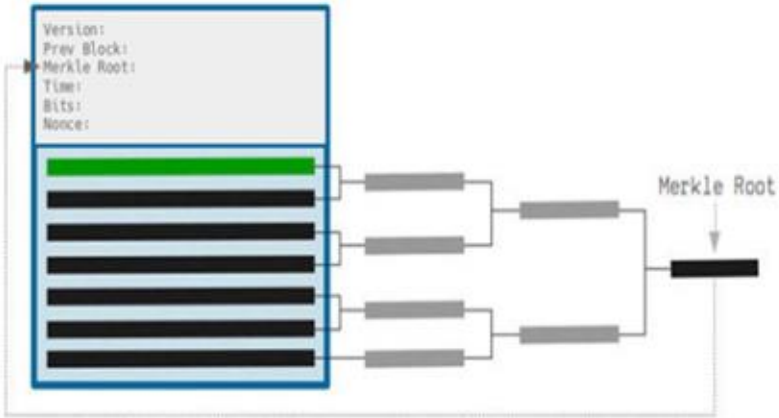
- SPV node will have less than a kilobyte of data for the block header and merkle path, an amount of data that is more than a thousand times less than a full block (1-4 MB).

Bitcoin: Transaction



Transactions Initiated by a Node





- Transactions are the most essential part of BTC system.
- Transactions are data structures that encode the transfer of value between participants in the BTC system.
- Each transactions is a public entry in BTC blockchain.

Independent Verification of Transaction

- Every node before forwarding transactions to its neighbors, it will first verify the transaction, which ensures that only valid transactions are propagated across the network, while invalid transactions are discarded at the first node that encounters them.
- Each node verifies every transaction against a long checklist of criteria:
  - The transaction's syntax and data structure must be correct
  - Neither lists of inputs or outputs are empty
  - The transaction size in bytes is less than MAX_BLOCK_SIZE
  - Each output value, as well as the total, must be within the allowed range of values (less than 21m coins, more than 0)
  - None of the inputs have hash=0, N=-1 (coinbase transactions should not be relayed)
  - nLockTime is less than or equal to INT_MAX

Independent Verification of Transaction

- The transaction size in bytes is greater than or equal to 100
- The number of signature operations contained in the transaction is less than the signature operation limit
- The unlocking script (scriptSig) can only push numbers on the stack, and the locking script (scriptPubkey) must match isStandard forms (this rejects “nonstandard” transactions)
- A matching transaction in the pool, or in a block in the main branch, must exist
- For each input, if the referenced output exists in any other transaction in the pool, reject this transaction
- For each input, look in the main branch and the transaction pool to find the referenced output transaction. If the output transaction is missing for any input, this will be an orphan transaction. Add to the orphan transactions pool, if a matching transaction is not already in the pool

Independent Verification of Transaction

- For each input, if the referenced output transaction is a coinbase output, it must have at least COINBASE_MATURITY (100) confirmations
- For each input, the referenced output must exist and cannot already be spent
- Using the referenced output transactions to get input values, check that each input value, as well as the sum, are in the allowed range of values (less than 21m coins, more than 0)
- Reject if the sum of input values < sum of output values
- Reject if transaction fee would be too low to get into an empty block
- The unlocking scripts for each input must validate against the corresponding output locking scripts

- By independently verifying each transaction as it is received and before propagating it, every node builds a pool of valid new transactions (the transaction pool), roughly in the same order.

Structure of the Regular and Generation Transaction

Size	Field	Description
32 bytes	Transaction Hash	Pointer to the transaction containing the UTXO to be spent
4 bytes	Output Index	The index number of the UTXO to be spent, first one is 0
1-9 bytes (VarInt)	Unlocking-Script Size	Unlocking-Script length in bytes, to follow
Variable	Unlocking-Script	A script that fulfills the conditions of the UTXO locking-script.
4 bytes	Sequence Number	Currently-disabled Tx-replacement feature, set to 0xFFFFFFFF

A regular transaction input

Size	Field	Description
32 bytes	Transaction Hash	All bits are zero: Not a transaction hash reference
4 bytes	Output Index	All bits are ones: 0xFFFFFFFF
1-9 bytes (VarInt)	Coinbase Data Size	Length of the Coinbase Data, from 2 to 100 bytes
Variable	Coinbase Data	Arbitrary Data used for extra nonce and mining tags In v2 blocks, must begin with block height
4 bytes	Sequence Number	Set to 0xFFFFFFFF

The generation transaction input

Structure of the Regular and Generation Transaction

- In a generation transaction, the first two fields are set to values that do not represent a UTXO reference. Instead of a “Transaction Hash”, the first field is filled with 32 bytes all set to zero.
- The “Output Index” is filled with 4 bytes all set to 0xFF (255 decimal).
- The “Unlocking Script” is replaced by coinbase data, an arbitrary data field used by the miners.

Coinbase Data

- Generation transactions do not have an unlocking script (a.k.a. scriptSig) field. Instead, this field is replaced by coinbase data, which must be between 2 and 100 bytes.
- Except for the first few bytes the rest of the coinbase data can be used by miners in any way they want; it is arbitrary data.
- In the genesis block, for example, Satoshi Nakamoto added the text “The Times 03/Jan/2009 Chancellor on brink of second bailout for banks” in the coinbase data, using it as a proof of the date and to convey a message. Currently, miners use the coinbase data to include extra nonce values and strings identifying the mining pool
- The first few bytes of the coinbase used to be arbitrary, but that is no longer the case. BIP0034), version-2 blocks (blocks with the version field set to 2) must contain the block height index as a script “push” operation in the beginning of the coinbase field.

Coinbase Data

- In block 277,316, the coinbase in the “Unlocking Script” or scriptSig field of the transaction input, contains the hexadecimal value 03443b04038584020 62f503253482f.
- The first byte, 03 instructs the script execution engine to push the next 3 bytes onto the script stack. The next 3 bytes, 0x443b04, are the block height encoded in little-endian format. Reverse the order of the bytes and the result is 0x043b44 which is 277,316 in decimal. The next few hexadecimal digits (03858402062) are used to encode an extra nonce. “The Extra Nonce or random value, used to find a suitable PoW solution.
- The final part of the coinbase data (2f503253482f) is the ASCII-encoded string, which indicates that the mining node that mined this block supports P2SH improvement defined in BIP0016.

Coinbase Data

- Those endorsing the BIP0017 implementation of P2SH were to include the string “p2sh/CHV” in their coinbase data. The BIP0016 was elected as the winner, and many miners continued including the string “/P2SH/” in their coinbase to indicate support for this feature.
- Constructing the Block Header: To construct the block header, the mining node needs to fill in six fields:

Size	Field	Description
4 bytes	Version	A version number to track software/protocol upgrades
32 bytes	Previous Block Hash	A reference to the hash of the previous (parent) block in the chain
32 bytes	Merkle Root	A hash of the root of the Merkle-Tree of this block's transactions
4 bytes	Timestamp	The approximate creation time of this block (seconds from Unix Epoch)
4 bytes	Difficulty Target	The Proof-of-Work algorithm difficulty target for this block
4 bytes	Nonce	A counter used for the Proof-of-Work algorithm

Coinbase Data

- To summarize all the transactions with a Merkle Tree, in order to add the Merkle Root to the block header. The generation transaction is listed as the first transaction in the block.
- If total transaction is odd the last last transaction is duplicated, each containing the hash of one transaction. The transaction hashes are then combined, in pairs, creating each level of the tree, until all the transactions are summarized into one node at the “root” of the tree. The root of the merkle tree summarizes all the transactions into a single 32 byte value.
- The mining node will then add a 4-byte timestamp, encoded as a Unix “Epoch” time-stamp, which is based on the number of seconds elapsed from January 1st, 1970, mid-night UTC/GMT.

Transaction Age, Fees, and Priority

- To construct the candidate block, a node selects transactions from the memory pool, by applying a priority metric to each transaction and adding the highest priority transactions first. Transactions are prioritized based on the “age” of the UTXO that is being spent in their inputs, allowing for old and high-value inputs to be prioritized over newer and smaller inputs.
- The priority of a transaction is calculated
  - Priority = Sum (Value of input * Input Age) / Transaction Size
- The age of a UTXO is the number of blocks that have elapsed since the UTXO was recorded on the blockchain

Transaction Age, Fees, and Priority

- The size of the transaction is measured in bytes, e.g. for a transaction to be considered “high priority” its priority must be greater than 57,600,000, which corresponds to one bitcoin (100m satoshis), aged one day (144 blocks) in a transaction of 250 bytes total size.

High Priority > 100,000,000 satoshis * 144 blocks / 250 bytes = 57,600,000

- The first 50 kilobytes of transaction space in a block are set aside for high priority transactions. A node will fill the first 50 kilobytes, prioritizing the highest priority transactions first, regardless of fee. This allows high-priority transactions to be processed even if they carry zero fees ?

Transaction Age, Fees, and Priority

- Node may choose to fill it with no-fee transactions. Some miners choose to mine transactions without fees on a best-effort basis. Other miners may choose to ignore transactions without fees?
- Transaction's priority depends on the age of its inputs, transactions remaining in the pool will age and therefore increase in priority. Eventually a transaction without fees may reach a high enough priority to be included in the block for free.
- Bitcoin transactions do not have an expiration time-out. A transaction that is valid now will be valid in perpetuity.
- The Generation Transaction - the first transaction added to the block is a special transaction, called a generation transaction or coinbase transaction.

Transaction Age, Fees, and Priority

- Node creates the generation transaction as a payment to his own wallet
- Unlike regular transactions, the generation transaction does not consume (spend) UTXO as inputs. Instead, it has only one input, called the coinbase, which creates bitcoin from nothing.
- The generation transaction has one output, payable to the miner's own bitcoin address.
- A node first calculates the total amount of transaction fees by adding all the inputs and outputs of the total transactions that suppose to be added in the block. The fees are calculated as:

  Total Fees = Sum(Inputs) - Sum(Outputs)

Transaction Priority System: Fee Market

- In BTC users compete for limited block space. Miners prioritize transactions based on factors like transaction fees and size, with transactions with higher fees usually completed faster since miners prioritize them to maximize their profits.
- Transaction fees are measured in "satoshis per virtual byte" (sat/vB), where virtual bytes represent the data size of your transaction
- Higher sat/vB rates = faster confirmation times
- Dust

Transaction Priority System: Fee Market

- What Determines Transaction Size: A transaction's size is not determined by how much BTC it transfers, but by how many inputs, outputs and signatures it has, and which address type it uses.
- Inputs = the sources of Bitcoin you're spending viz. combining coins from different pockets.
- Outputs = the destinations (recipient address plus change back to you)
- Address types = newer address formats like SegWit create smaller transactions

Transaction Priority System: Fee Market

- Prioritization Factors: Miners consider several factors when selecting transactions:
- Fee rate (sat/vB) – the primary factor determining priority.
- Transaction age – older transactions may get slight preference.
- Child-pays-for-parent (CPFP): When a transaction is stuck due to low fees, the recipient can create a new transaction (the 'child') that spends from the stuck transaction (the 'parent') using a much higher fee rate. Since miners want to collect the higher fee from the child transaction, they must process both the parent and child transactions together, effectively speeding up the stuck parent transaction.
- Replace-by-fee (RBF): Another method that allows users to increase fees on unconfirmed transactions to speed up transaction times.

Transaction Priority System: Fee Market

- Network Congestion Effects viz. during busy periods (like market rallies or major news events).
- Mempool size increases as more transactions compete for block space.
- Required fees rise as users bid against each other for faster confirmation.
- Lower-fee transactions get delayed.
- Confirmation times become unpredictable.

Difficulty Target

- The node then fills in the difficulty target, which defines the required PoW difficulty to make this a valid block.
- The difficulty target is encoded as a one-byte exponent, followed by a 3 byte mantissa (coefficient). In block 277,316, for example, the difficulty bits value is 0x1903a30c. The first part 0x19 is a hexadecimal exponent, while the next part 0x03a30c is the coefficient
- The final field is the nonce, which is initialized to zero.
- With all the other fields filled, the block header is now complete and the process of mining can begin. The goal is now to find a value for the nonce that results in a block header hash that is less than the difficulty target.
- To produce a hash result matching a specific target, is to try again and again, randomly modifying the input until the desired hash result appears by chance.

Difficulty Representation

- In this block, therefore, the exponent is 0x19 and the coefficient is 0x03a30cThe formula to calculate the difficulty target from this representation is:
  - target = coefficient * 2^(8 * (exponent - 3))
  - Using that formula, and the difficulty bits value 0x1903a30c, we get:
  - target = 0x03a30c * $2^{(0x08 * (0x19 - 0x03))}$ = 0x03a30c * $2^{(0x08 * 0x16)}$ = 0x03a30c * 2^{0xB0}
  - which in decimal is target = 238,348 * 2^{176}
  - =22,829,202,948,393,929,850,749,706,076,701,368,331,072,452,018,388,575,715,328
- Switching back to hexadecimal:
  - target = 0x00000000000003A30C000000000000000000000000000000000000000000

Proof-of-Work Algorithm

- To make a challenge out of this algorithm, let's set an arbitrary target: find a phrase that produces a hexadecimal hash that starts with a zero. In numerical terms, that means finding a hash value that is less than 0x1000000000000000000000000000000000000000000000000000000000000000.
- If we decrease the target, the task of finding a hash that is less than the target becomes more and more difficult.
- The miner will modify the nonce and try again to achieve the target.
- Suppose the network is attempting to find a block whose header hash is less than 00000000000004c296e6376db3a241271f43fd3f5de7ba18986e517a243baa7.

Proof-of-Work Algorithm

- More zeros more difficult to find a valid hash. It will take on average more than 150 quadrillion hash calculations per second for the network to discover the next block.
- Seems like an impossible task, but fortunately the network is bringing 100 Peta Hashes per second of processing power to bear, which will be able to find a block in about 10 minutes on average.
- The block contains the difficulty target, in a notation called “difficulty bits” or just “bits”, which in block 277,316 has the value of 0x1903a30c. This notation expresses the difficulty target as a coefficient/exponent format, with the first two hexadecimal digits for the exponent and the next six hex digits as the coefficient.

New Difficulty

- A valid block is one that has a block header hash that is less than the target. In binary that number would have more than the first 60 bits set to zero. With this level of difficulty, a single miner processing 1 trillion hashes per second would only find a solution once every 8,496 blocks or once every 59 days, on average.
- Difficulty Target and Re-Targeting-Why is the difficulty adjustable, who adjusts it and how?
- Bitcoin's blocks are generated every 10 minutes, on average. To keep the block generation time at 10 minutes, the difficulty of mining must be adjusted to account for these changes. In fact, difficulty is a dynamic parameter that will be periodically adjusted to meet a 10-minute block target.
- The difficulty of finding a bitcoin block is approximately 10 minutes of processing for the entire network, based on the time it took to find the previous 2016 blocks, adjusted every 2016 blocks.

New Difficulty

- In simple terms, the difficulty target is set to whatever mining power will result in a 10-minute block interval.If the network is finding blocks faster than every 10 minutes, the difficulty increases. If block discovery is slower than expected, the difficulty decreases.
- The equation can be summarized as:
  - New Difficulty = Old Difficulty * (Actual Time of Last 2016 Blocks / 20160 minutes)
- To avoid extreme volatility in the difficulty, the retargeting adjustment must be less than a factor of four per cycle. If the required difficulty adjustment is greater than a factor of four, it will be adjusted by the maximum and not more.
- The target difficulty is independent of the number of transactions or the value of transactions.
- The primary influence on the mining market is the price of one kilowatt-hour in BTC, which determines the profitability of mining

Mining

- Mining is a process by which new BTC is added/generated to the money supply.
- Mining adds new blocks to the block chain, making transaction history hard to modify i.e., it secure the system against fraudulent transactions or spending the same amount more than once (double-spending).
- Mining is the main process of the decentralized clearinghouse, by which transactions are validated and cleared without central authority.
- Miners provide processing power in exchange for the opportunity to get BTC by validating new transactions and record them on the global ledger.
- Miners receive two types of reward: new coins created with each new block and transaction fees from all the transactions included in the block.

Mining

- Approximately every four years (or precisely every 210,000 blocks), the reward gets halved.
- It started at 50 bitcoin per block in January of 2009 and since April 2024 it is 3.125. In 2140 all BTC (20.99999998 million) will be issued, thereafter, no new bitcoins will be issued.
- After 2140, miners will be rewarded solely through the transaction fees.
- Deflationary Money: Consequence of a fixed and diminishing monetary issuance is that the currency will tend to be inherently deflationary, i.e. the value of BTC will be appreciated due to a mismatch in supply and demand that drives up the value of a currency.

Mining Node

- Solo, Pool, Managed Pool, P2P Pool
- Some of the nodes on the network are specialized nodes called miners, a full BTC node.
- The arrival of a new block has special significance for a mining node i.e. competition among miners effectively ends with the propagation of a new block which acts as an announcement of a winner.
- The end of one round of a competition is also the beginning of the next round. The new block is not just a checked flag, marking the end of the race, it is also the starting pistol starting the race for the next block.

Validating a New Block

- The third step in bitcoin's consensus mechanism is independent validation of each new block by every node on the network.
- Each node performs a series of tests to validate it before propagating it to its peers, which ensures that only valid blocks are propagated on the network.
- The independent validation also ensures that miners who act honestly get their blocks incorporated in the blockchain, thus earning the reward.

Validating a New Block

- When a node receives a new block, it will validate the block by checking it against a long list of criteria that must all be met; otherwise the block is rejected.
  - The block data structure is syntactically valid
  - The block header hash is less than the target difficulty
  - The block timestamp is less than two hours in the future (allowing for time errors)
  - The block size is within acceptable limits
  - The first transaction is a coinbase generation transaction
  - All transactions within the block are valid using the transaction checklist

Validating a New Block

- The independent validation of each new block by every node on the network ensures that the miners can't cheat.
- Why doesn't the miner write themselves a transaction for a thousand bitcoin instead of the correct reward? Because every node validates blocks according to the same rules.

Assembling and Selecting Chains of Blocks

- Once a node has validated a new block, it will then attempt to assemble a chain, by connecting the block to the existing blockchain.
- Nodes maintain three sets of blocks: those connected to the main blockchain, those that form branches off the main blockchain (secondary chains) and finally blocks that do not have a known parent in the known chains (orphans).
- The “main chain” at any time is whichever chain of blocks has the most cumulative difficulty associated with it. Under most circumstances this is also the chain with the most blocks in it, unless there are two equal length chains and one has more PoW.

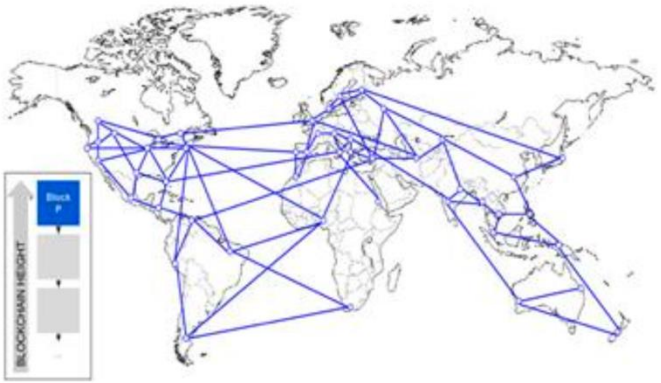
Assembling and Selecting Chains of Blocks

- The main chain will also have branches with blocks that are “siblings” to the blocks on the main chain. These blocks are valid but not part of the main chain. They are kept for future reference, in case one of those chains is extended to exceed the main chain in difficulty.
- The node will attempt to find that parent in the existing blockchain.
- The tip of the main chain will link the new block and extend that chain.
- The new block extends a chain that is not the main chain. In that case, the node will attach the new block to the secondary chain it extends and then compare the difficulty of the secondary chain to the main chain. If the secondary chain has more cumulative difficulty than the main chain, the node will reconverge on the secondary chain, meaning it will select the secondary chain as its new main chain, making the old main chain a secondary chain.

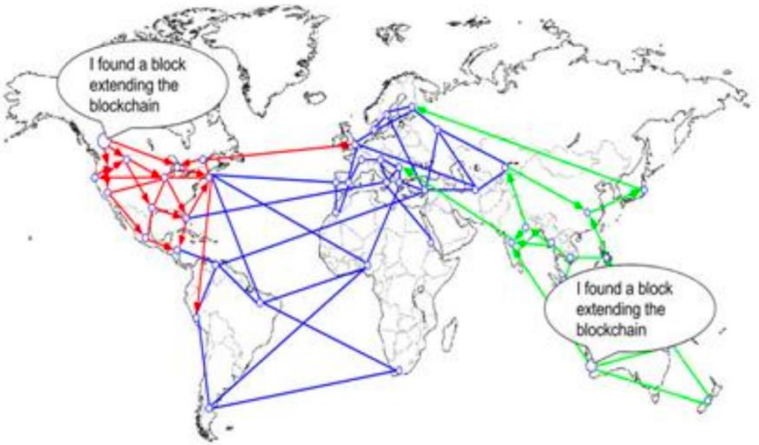
Assembling and Selecting Chains of Blocks

- By selecting the greatest-difficulty chain, all nodes eventually achieve network-wide consensus. Temporary discrepancies between chains are resolved eventually as more PoW is added, extending one of the possible chains.
- Mining nodes “vote” with their mining power by choosing which chain to extend by mining the next block. When they mine a new block and extend the chain, the new block itself represents their vote

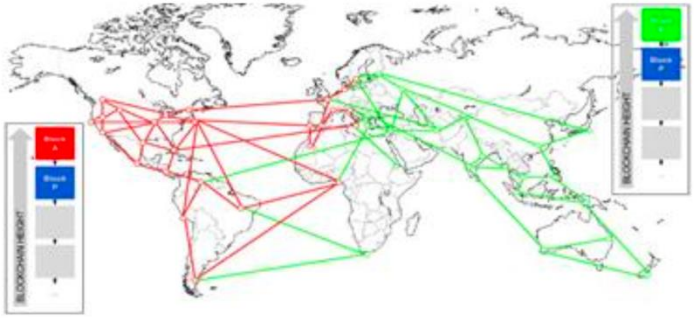
Block Forks



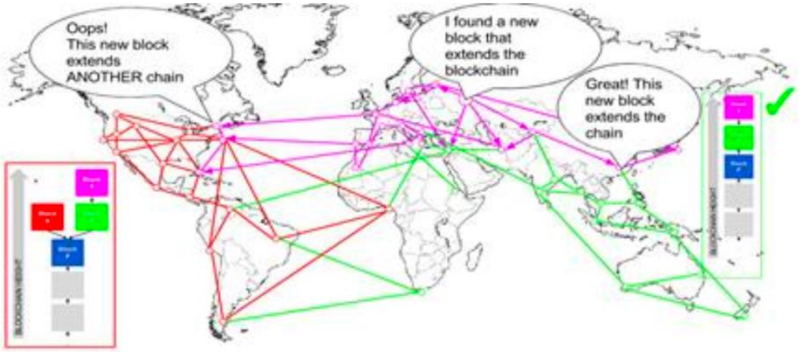
A blockchain fork event: Before the Fork



A blockchain fork event: Two blocks found simultaneously



A blockchain fork event: wo blocks propagate, splitting the network



A blockchain fork event: The network reconverges on a new longest chain

Thanks!!!

Queries?